Lokakarya

Coding Python

Dasar-dasar Logika Python

Disusun Oleh
Narasumber UPT TIKP



Tujuan

- Memahami masukan pengguna dan memproses data
- Melakukan operasi matematika dalam Python
- Menggunakan pernyataan kondisional untuk membuat keputusan
- Menerapkan operator logika untuk kondisi kompleks
- Menulis program interaktif dengan aplikasi dunia nyata

Apa itu Python

Python adalah bahasa pemrograman tingkat tinggi yang mudah ditafsirkan, dikenal karena kesederhanaan dan keterbacaannya. Bahasa ini banyak digunakan di berbagai bidang seperti pengembangan web, kecerdasan buatan, ilmu data, otomatisasi, dan robotika. Sintaks Python yang mudah dipahami menjadikannya pilihan yang sangat baik bagi pemula, sementara pustakanya yang canggih memungkinkan programmer berpengalaman untuk membangun aplikasi kompleks secara efisien.

Mengapa Belajar Python

- **Kesederhanaan** – Kode Python mudah ditulis dan dipahami, membuatnya ideal untuk pemula.
- **Fleksibilitas** – Python dapat digunakan untuk pengembangan web, pembelajaran mesin, otomatisasi, dan banyak lagi.
- **Dukungan Komunitas Besar** – Komunitas pengembang yang luas berkontribusi pada Python, memastikan pembaruan dan dukungan yang berkelanjutan.
- **Pustaka yang Luas** – Python menawarkan koleksi pustaka yang kaya seperti NumPy, Pandas, dan Matplotlib untuk berbagai aplikasi.

beberapa aspek penting sintaksis Python:

A. Sensitivitas Huruf Besar/Kecil

B. Indentasi dalam Python

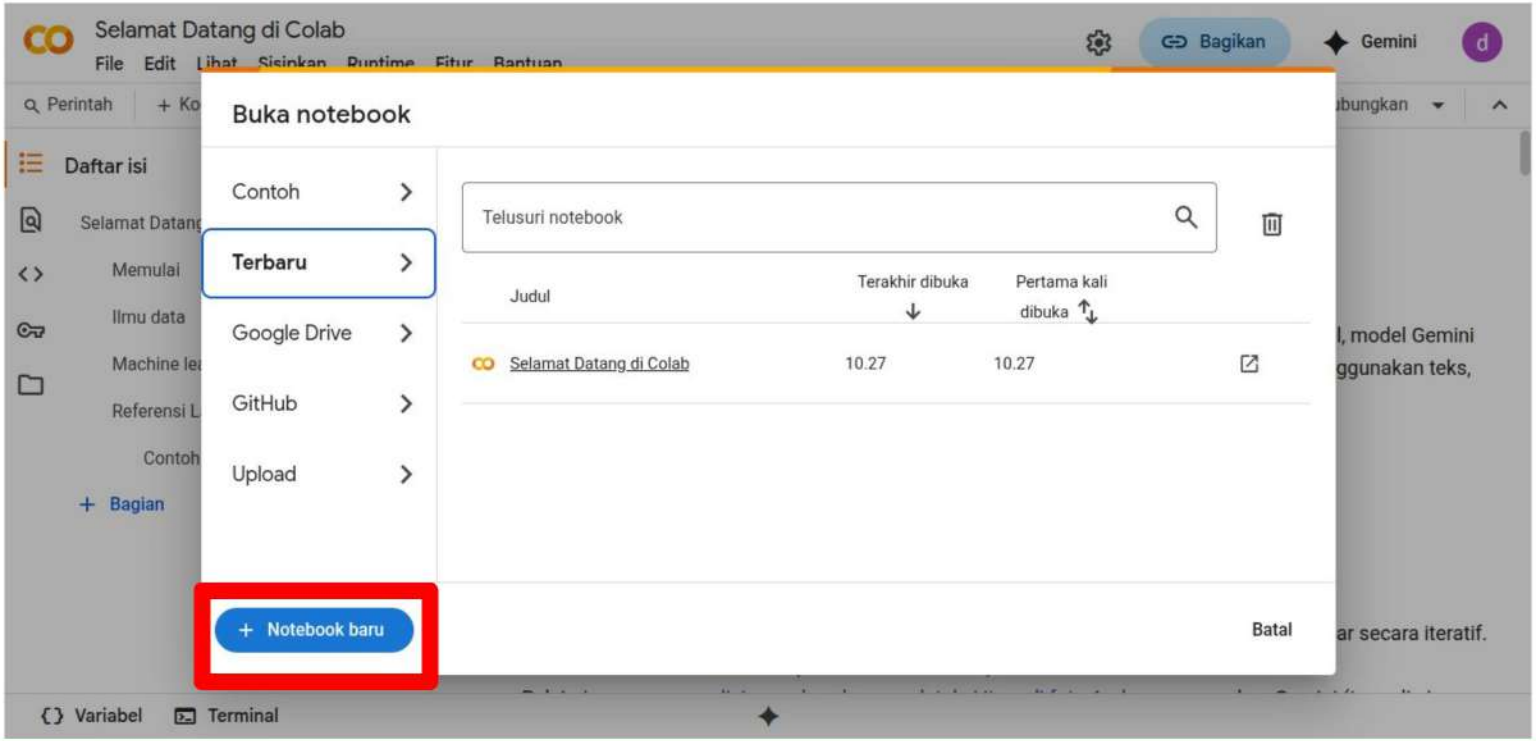
C. Menulis Komentar dalam Python

Akses Google Collabs

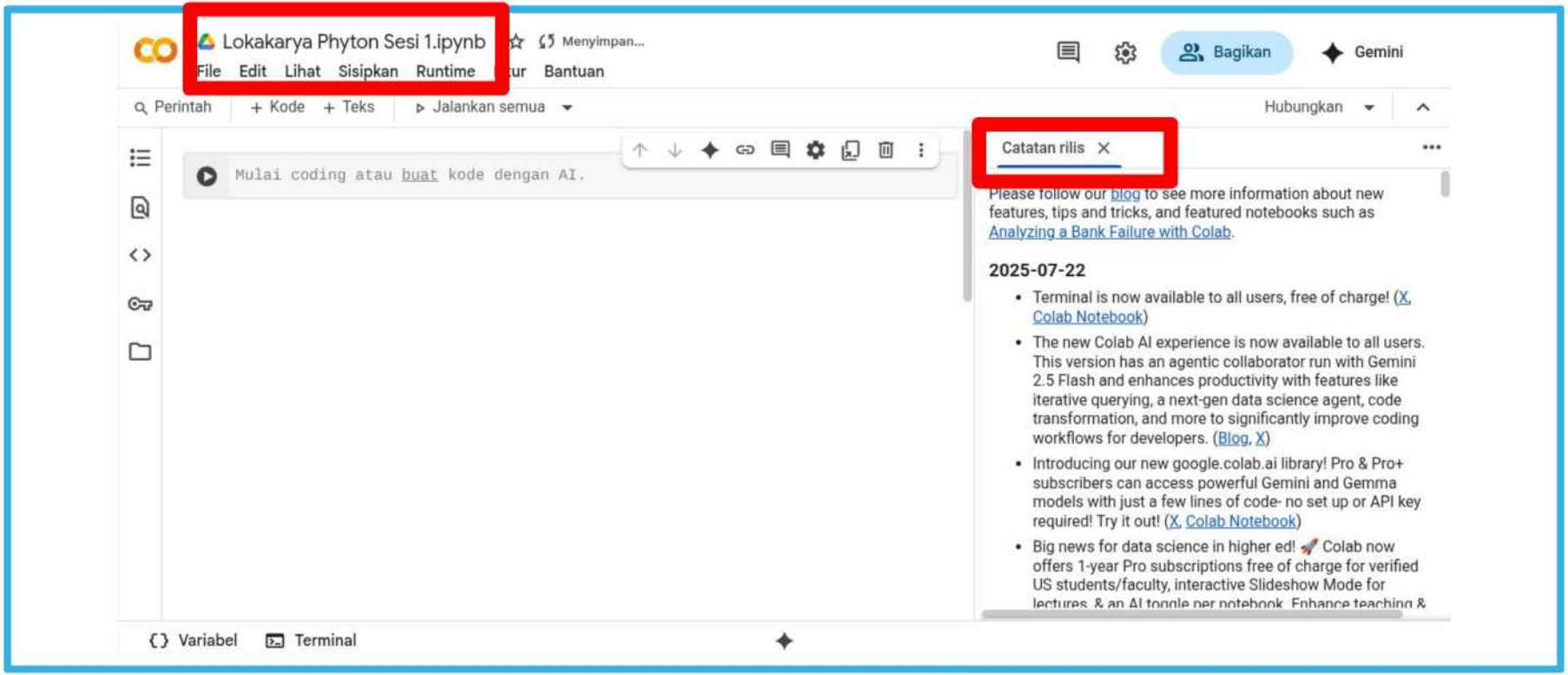


https://colab.research.google.com/?authuser=1

Klik Notebook Baru



Ganti Nama File dan Tutup Catatan



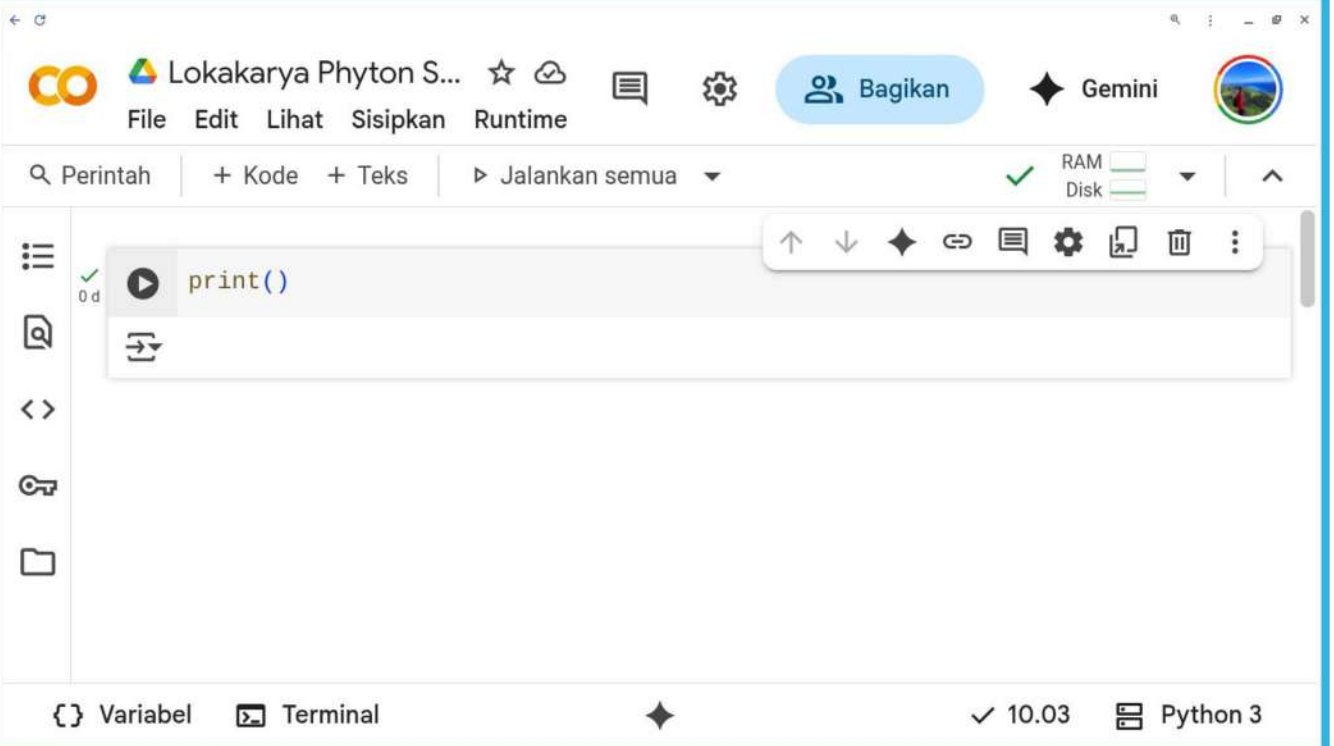
Case Sensitivity

Huruf besar-kecil `print()` dan `Print()` huruf kecil diperlakukan berbeda.

Fungsi yang benar selalu `print()`

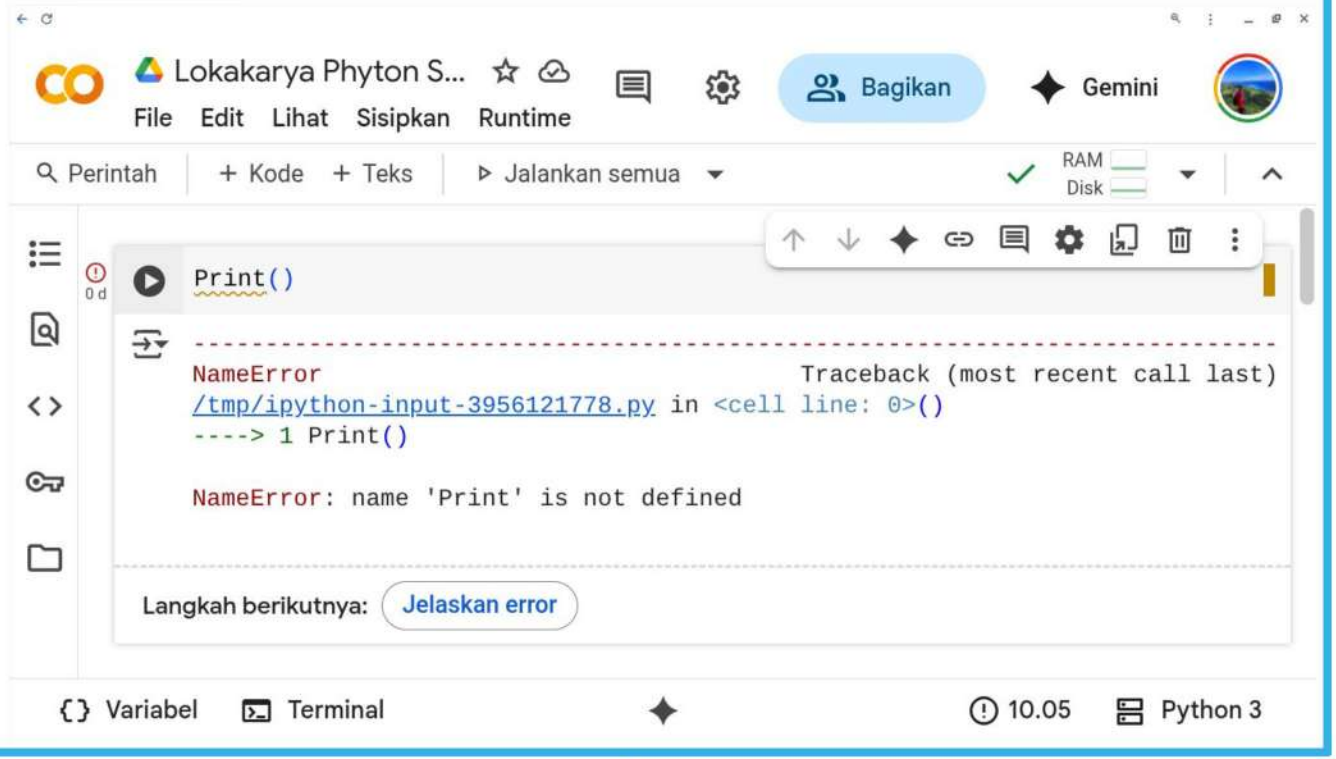
Case Sensitivity

Fungsi yang benar



Case Sensitivity

Fungsi yang salah



Indentasi dalam Python

Tidak seperti bahasa lain yang menggunakan tanda kurung `{}`, Python menggunakan indentasi (spasi atau tab) untuk mendefinisikan blok kode.

Contoh: Indentasi yang Benar

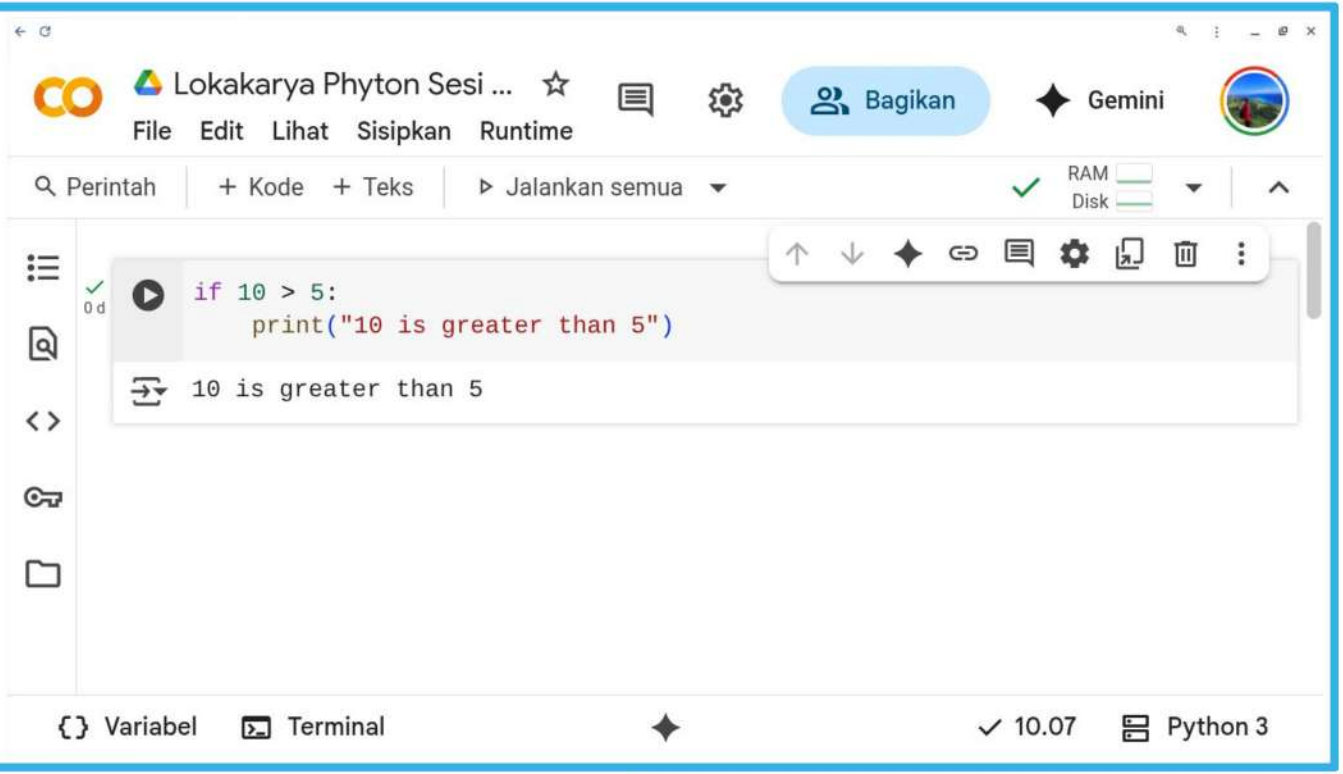
```
if 10 > 5:
    print("10 is greater than 5")
```

Indentasi yang Salah (Akan Menyebabkan Kesalahan)

```
if 10 > 5:
print("10 is greater than 5")  # No indentation leads to an error
```

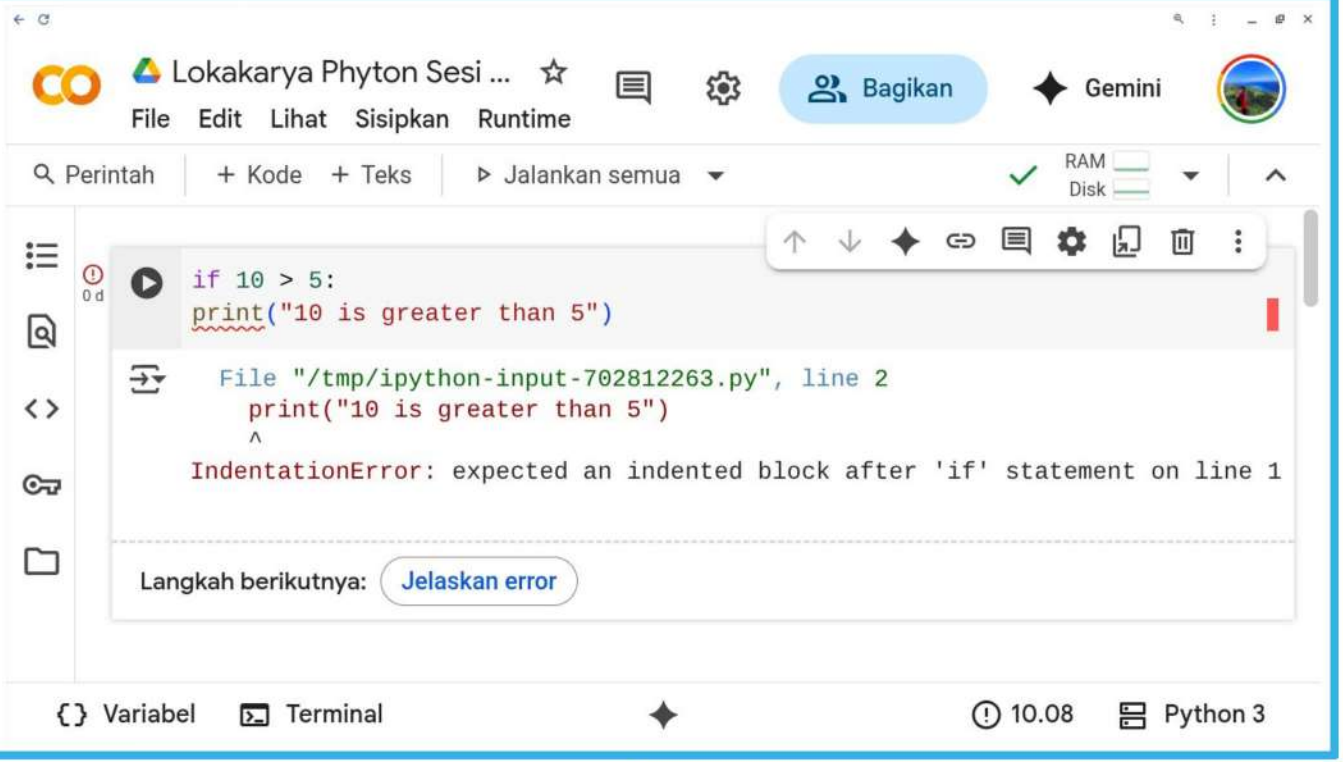
Indentasi dalam Python

Indentasi yang Benar



Indentasi dalam Python

Indentasi yang Salah



Menulis Komentar dalam Python

Komentar digunakan untuk menjelaskan kode dan diabaikan selama eksekusi.

- **Komentar satu baris:**

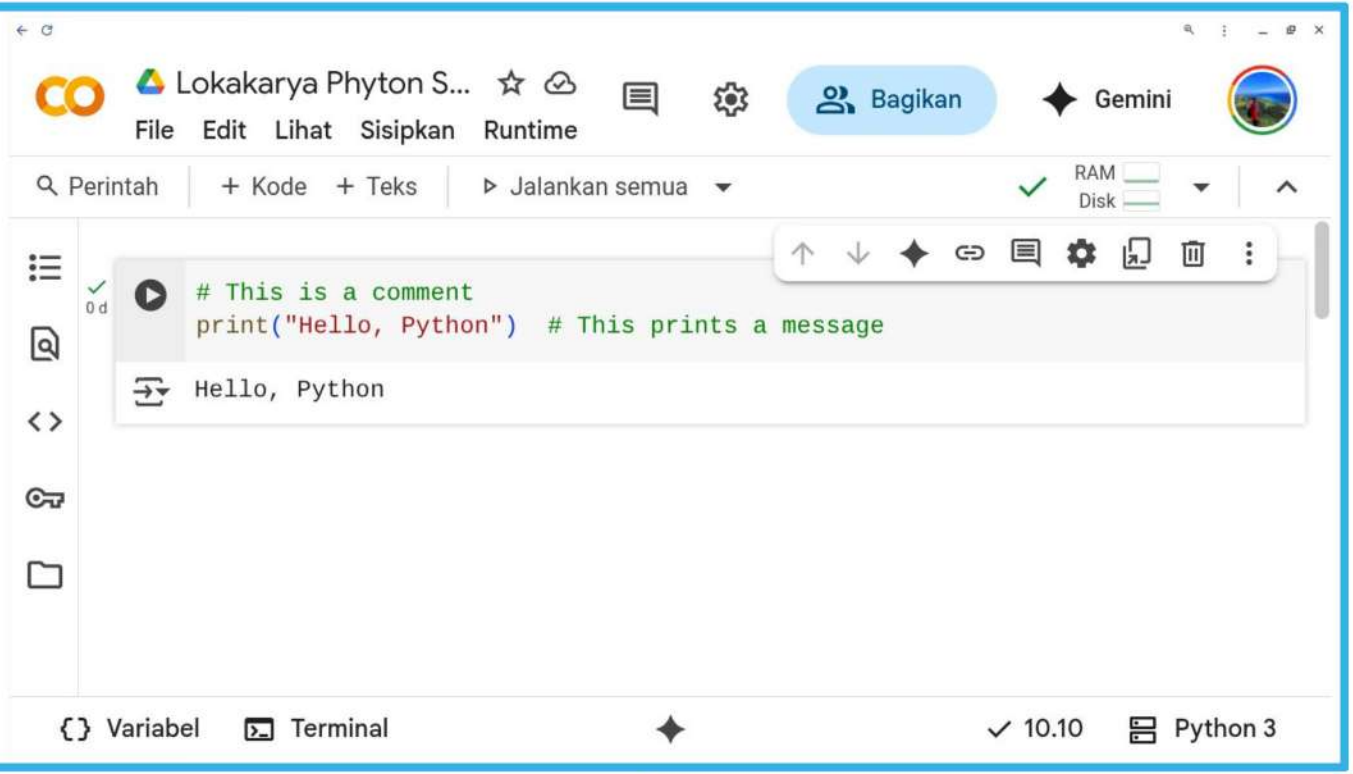
```
# This is a comment
print("Hello, Python")   # This prints a message
```

- **Komentar multi-baris:**

```
"""
This is a
multi-line comment
"""
print("Python is fun")
```

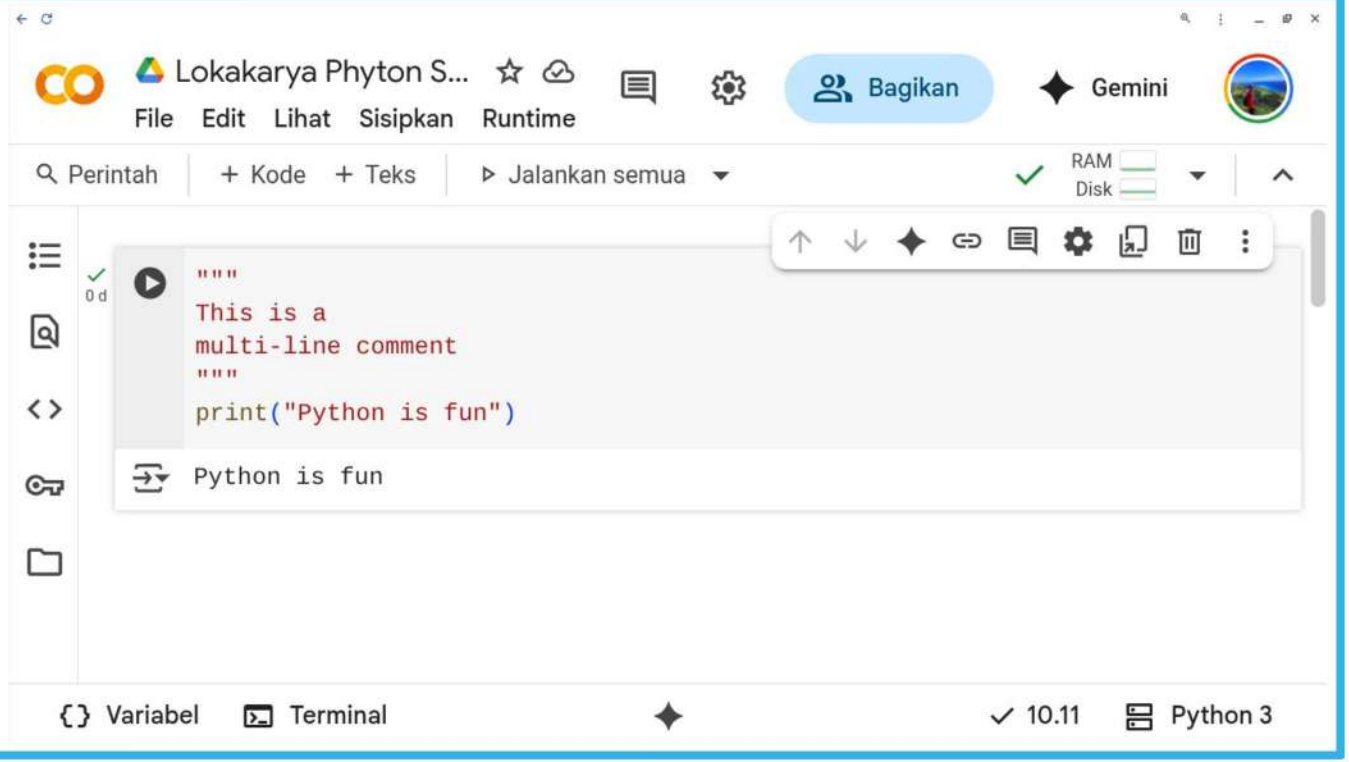
Menulis Komentar dalam Python

Komentar
Dalam
Phyton



Menulis Komentar dalam Python

Komentar Multi Baris Dalam Phyton



Input Pengguna dan Operasi Matematika

A. **Mengambil Input Pengguna dalam Python**

B. **Melakukan Operasi Matematika dalam Python**

Mengambil Input Pengguna dalam Python

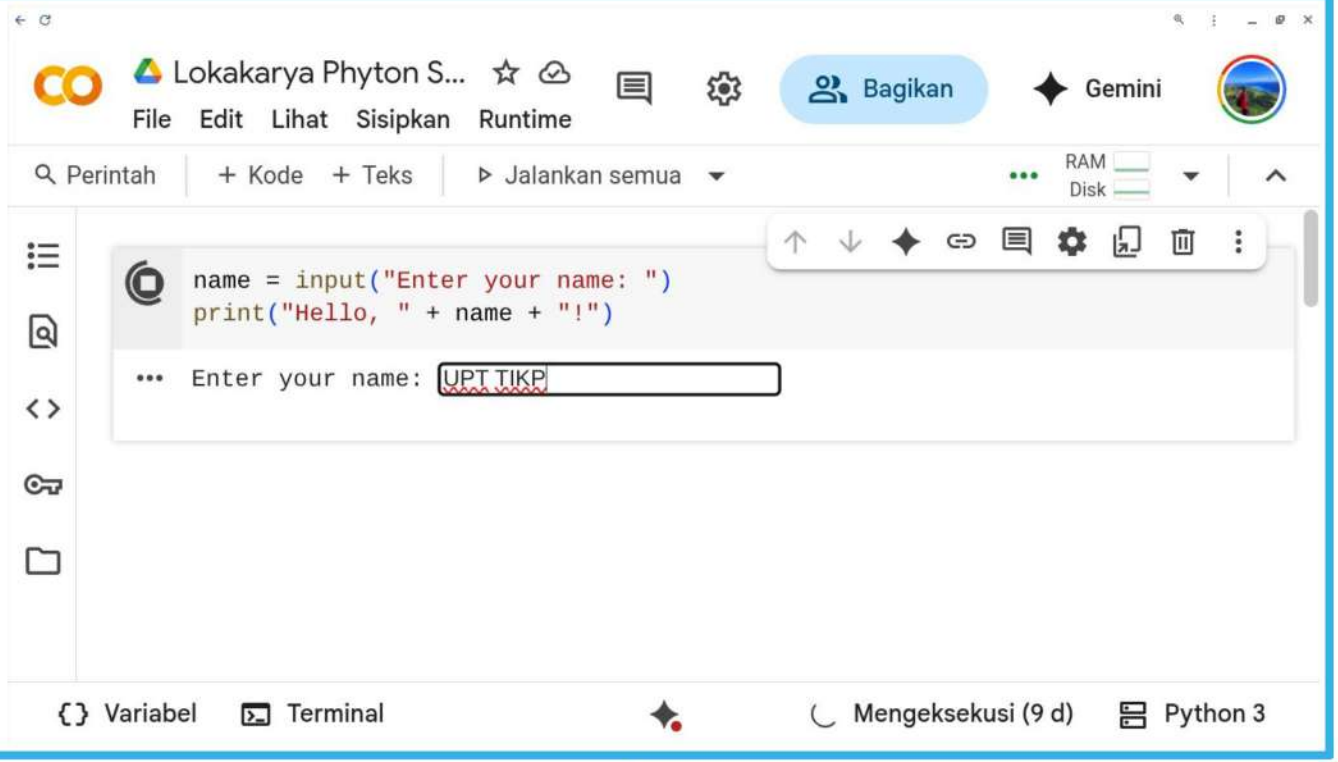
Python memungkinkan pengguna untuk memberikan masukan menggunakan `input()` fungsi tersebut.

Contoh: Mendapatkan Nama Pengguna

```
name = input("Enter your name: ")
print("Hello, " + name + "!")
```

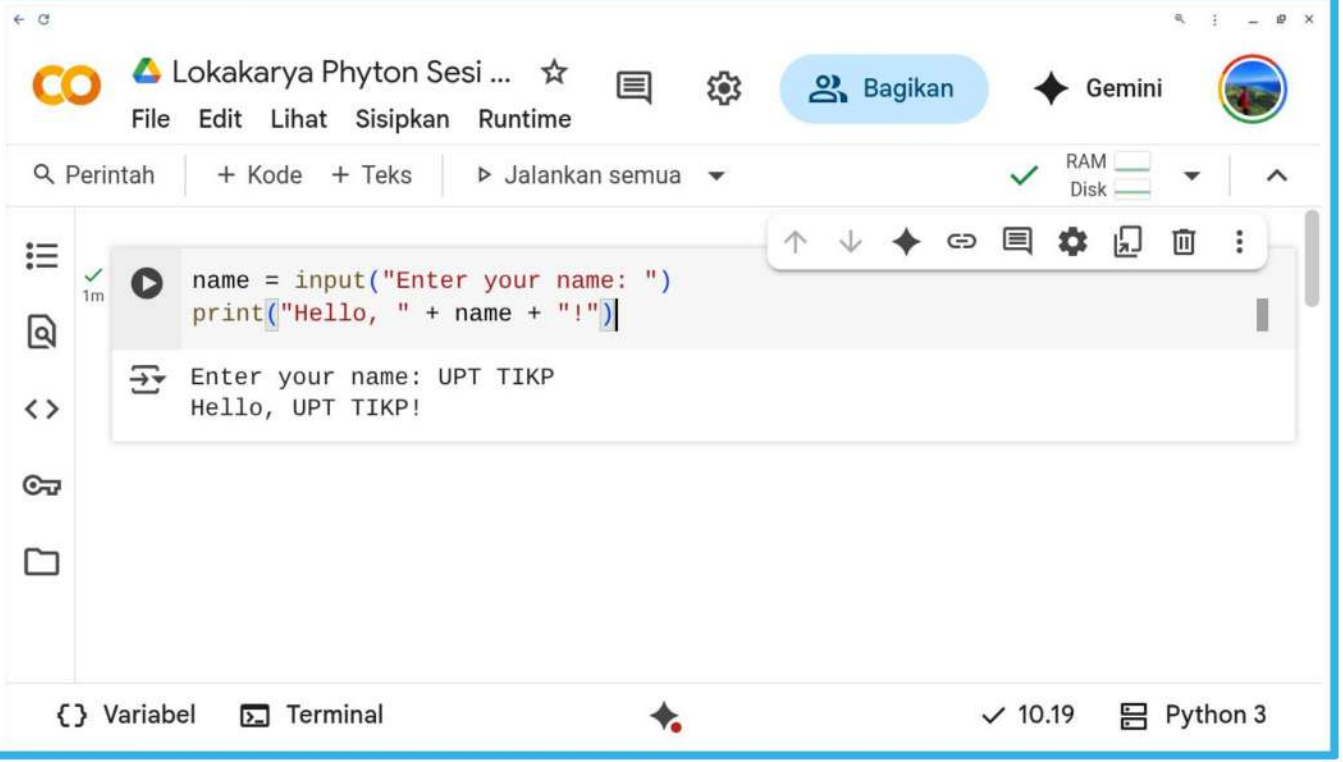
Mengambil Input Pengguna dalam Python

Mendapatkan Nama Pengguna



Mengambil Input Pengguna dalam Python

Mendapatkan Nama Pengguna



Explanation

- `input()` meminta pengguna untuk memasukkan data.
- Nilai yang dimasukkan disimpan dalam variabel `name`.
- Operator `+` digunakan untuk menggabungkan string dan menampilkan pesan yang dipersonalisasi.

Melakukan Operasi Matematika dalam Python

Matematika merupakan bagian fundamental dari pemrograman, dan Python menyediakan beragam operator untuk melakukan perhitungan aritmatika. Operasi-operasi ini membentuk fondasi untuk mengembangkan aplikasi dalam analisis data, pengembangan gim, kecerdasan buatan, dan banyak lagi.

Operator Aritmatika dalam Python

Operator	Simbol	Contoh	Keluaran
Tambahan	+	5 + 3	8
Pengurangan	-	10 - 4	6
Perkalian	*	6 * 3	18

Operator Aritmatika dalam Python

Operator	Simbol	Contoh	Keluaran
Pembagian	`/`	`15 / 3`	`5.0`
Pembagian Habis	`//`	`17 // 3`	`5`
Modulus	`%`	`17 % 3`	`2`
Eksponensiasi	`**`	`2 ** 3`	`8`

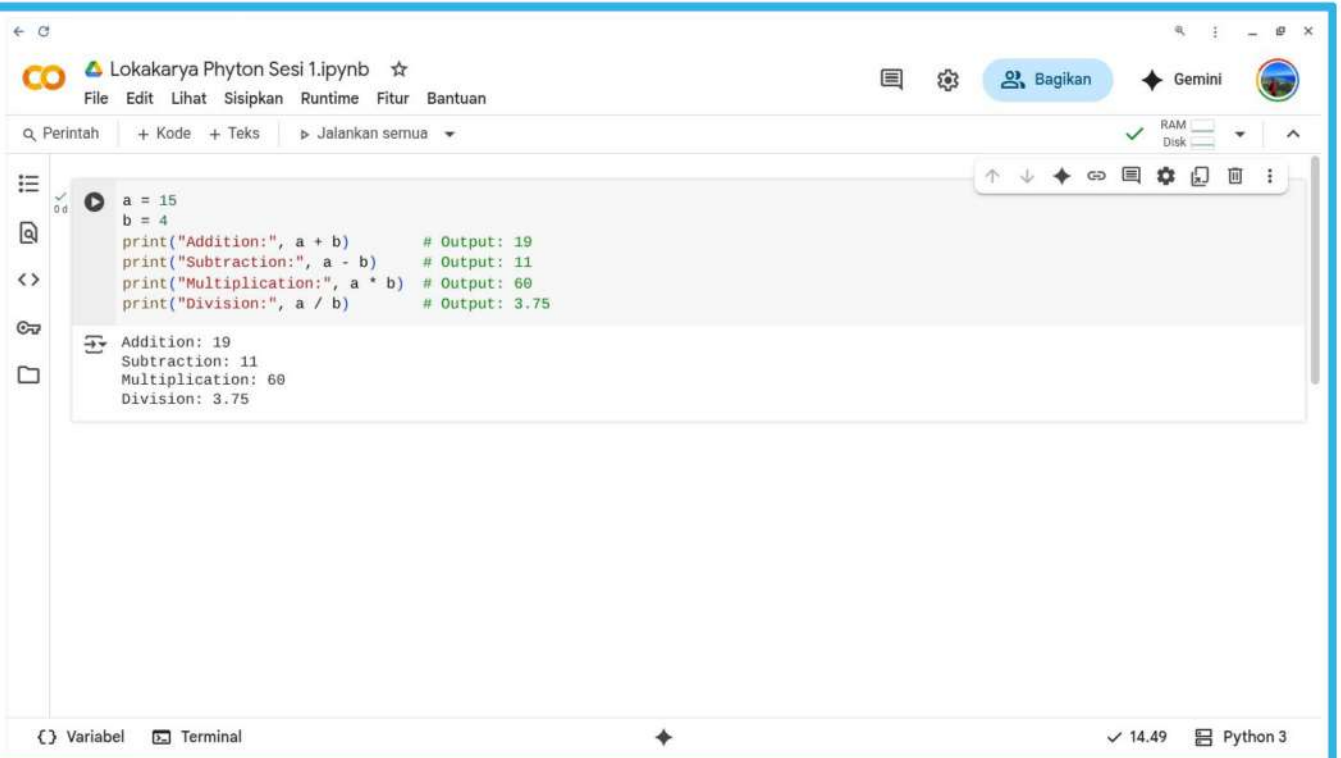
Contoh Aritmatika dalam Python

Mari kita telusuri cara kerja operator ini melalui contoh.

```
a = 15
b = 4
print("Addition:", a + b)          # Output: 19
print("Subtraction:", a - b)       # Output: 11
print("Multiplication:", a * b)    # Output: 60
print("Division:", a / b)          # Output: 3.75
```

Contoh Aritmatika dalam Python

Operasi Matematika



Pernyataan Kondisional dan Operator Logika

Pernyataan kondisional memungkinkan program untuk membuat keputusan berdasarkan kondisi tertentu. Dalam Python, pernyataan ini mengontrol alur eksekusi, menjadikan program interaktif dan dinamis.

A conditional statement checks whether a certain condition is true or false. Based on the result, different blocks of code execute.

Apa itu Pernyataan Kondisional?

Pernyataan kondisional memeriksa apakah suatu kondisi tertentu benar atau salah. Berdasarkan hasilnya, berbagai blok kode akan dieksekusi.

Jenis Pernyataan Kondisional Dalam Phyton

```
if pernyataan

if-else pernyataan

if-elif-else pernyataan

Pernyataan kondisional bersarang
```

Pernyataan if

Pernyataan tersebut `if` mengeksekusi blok kode hanya jika kondisi yang diberikan adalah `True`.

Sintaksis:

```
if condition:
    # Code to execute if the condition is true
```

Contoh Penggunaan Pernyataan if

Memeriksa Usia untuk Kelayakan Memilih

```
age = int(input("Enter your age: "))

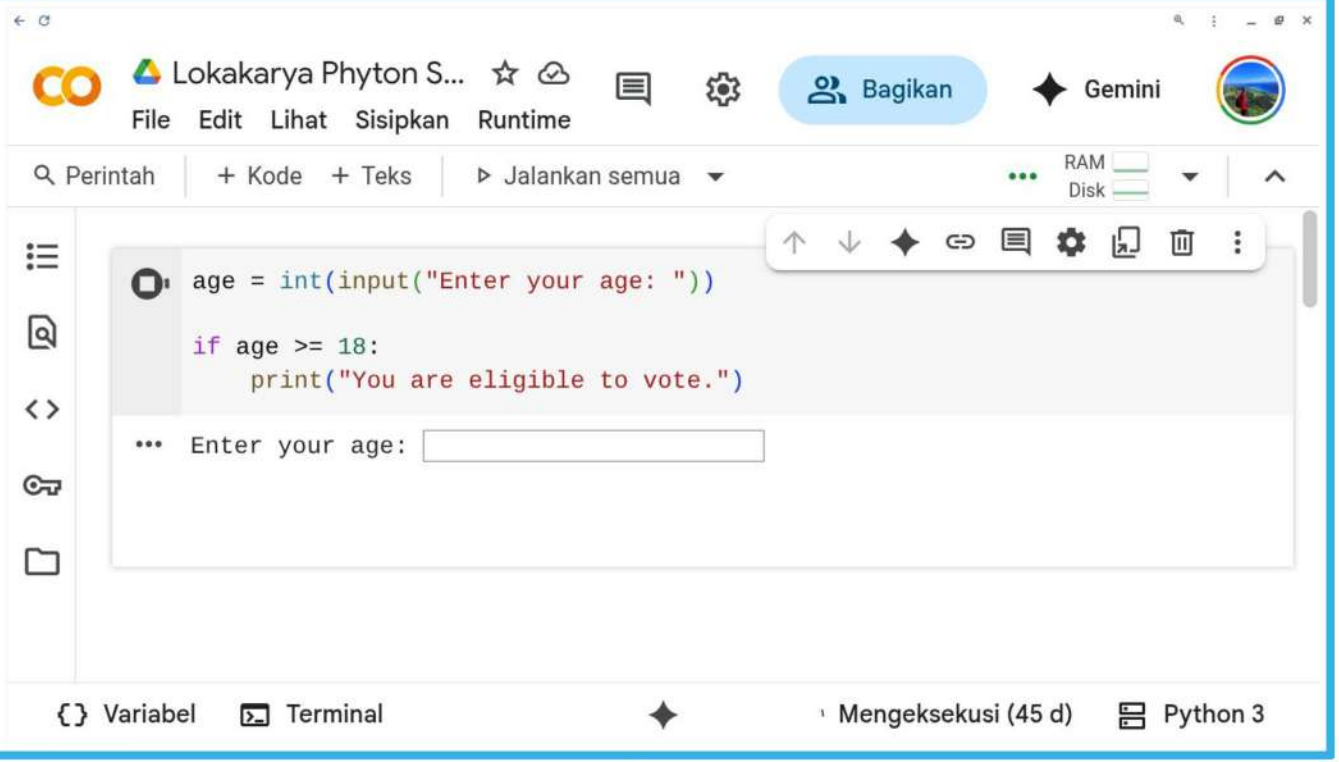
if age >= 18:
    print("You are eligible to vote.")
```

Penjelasan:

- Program menanyakan usia pengguna.
- Jika usianya **18 tahun atau lebih** , akan dicetak`"You are eligible to vote."`
- Jika usianya di bawah 18 tahun, tidak terjadi apa-apa.

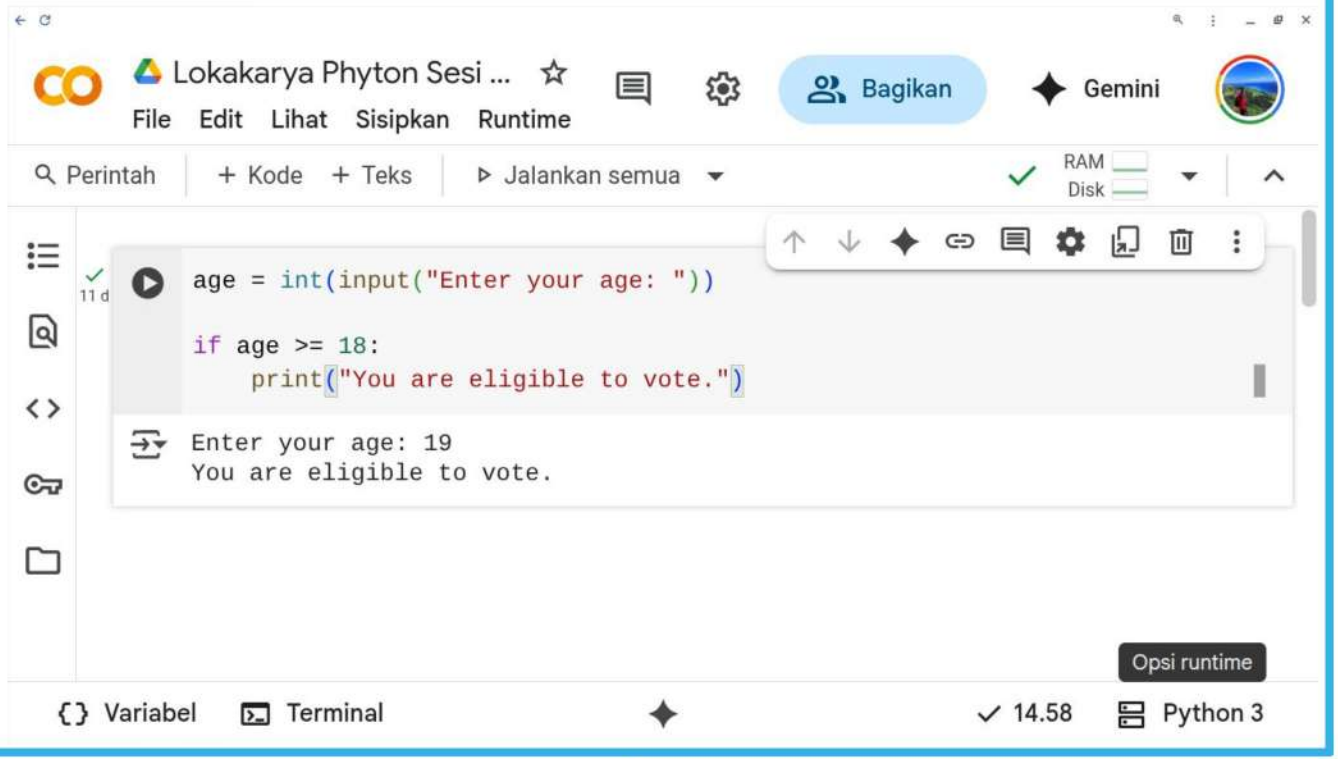
Contoh Penggunaan Pernyataan if

Pengecekan Pemenuhan Syarat Usia



Contoh Penggunaan Pernyataan if

Respon ketika memenuhi syarat



Pernyataan if-else

Pernyataan tersebut `if-else` menyediakan blok kode alternatif untuk dieksekusi jika kondisinya adalah `False`.

Sintaksis:

```
if condition:
    # Code if condition is true
else:
    # Code if condition is false
```

Contoh Penggunaan Pernyataan if-else

Pengecekan Angka Genap atau Ganjil:

```
num = int(input("Enter a number: "))

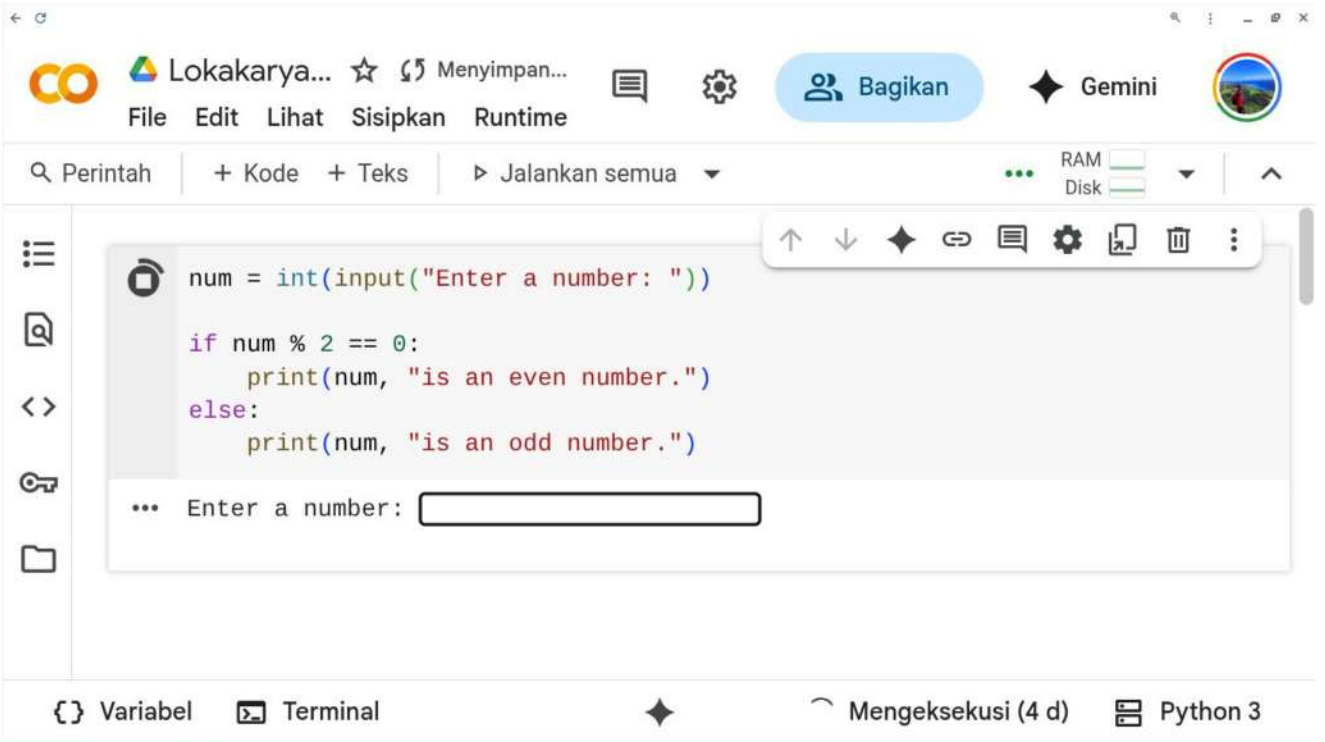
if num % 2 == 0:
    print(num, "is an even number.")
else:
    print(num, "is an odd number.")
```

Penjelasan:

- Program mengambil angka sebagai masukan.
- Ia memeriksa apakah sisa pembagian dengan 2 adalah nol (`num % 2 == 0`).
- Jika `True`, maka akan dicetak `"even number"`; jika tidak, maka akan dicetak `"odd number"`.

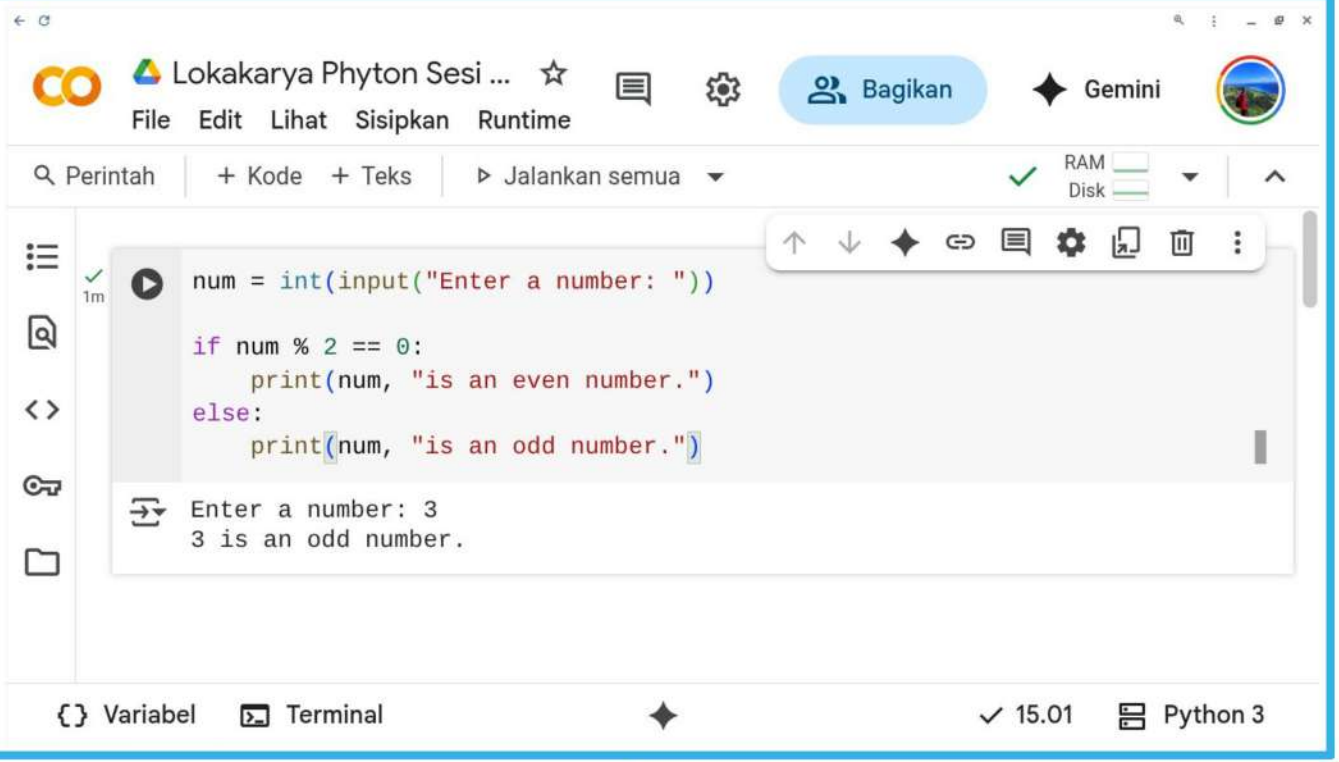
Contoh Penggunaan Pernyataan if-else

Pengecekan Nilai Ganjil atau Genap



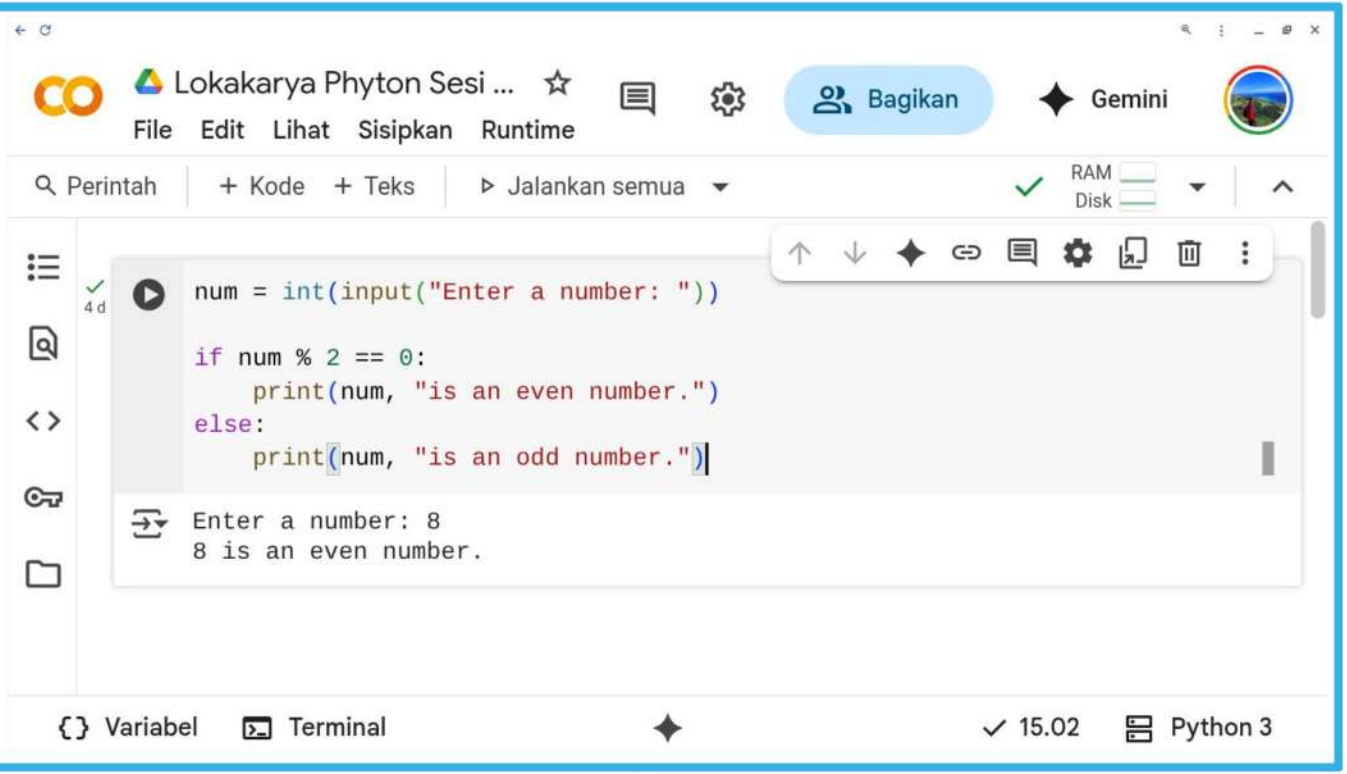
Contoh Penggunaan Pernyataan if-else

Hasil Pengecekan Nilai Ganjil



Contoh Penggunaan Pernyataan if-else

Hasil Pengecekan Nilai Genap



Pernyataan if-elif-else

Ketika beberapa kondisi perlu diperiksa, `if-elif-else` pernyataan digunakan. `elif` Kata kunci memungkinkan kondisi tambahan sebelum `else` pernyataan.

Sintaksis:

```
if condition1:
    # Code if condition1 is true
elif condition2:
    # Code if condition2 is true
else:
    # Code if none of the conditions are true
```

Contoh Penggunaan Pernyataan if-elif-else

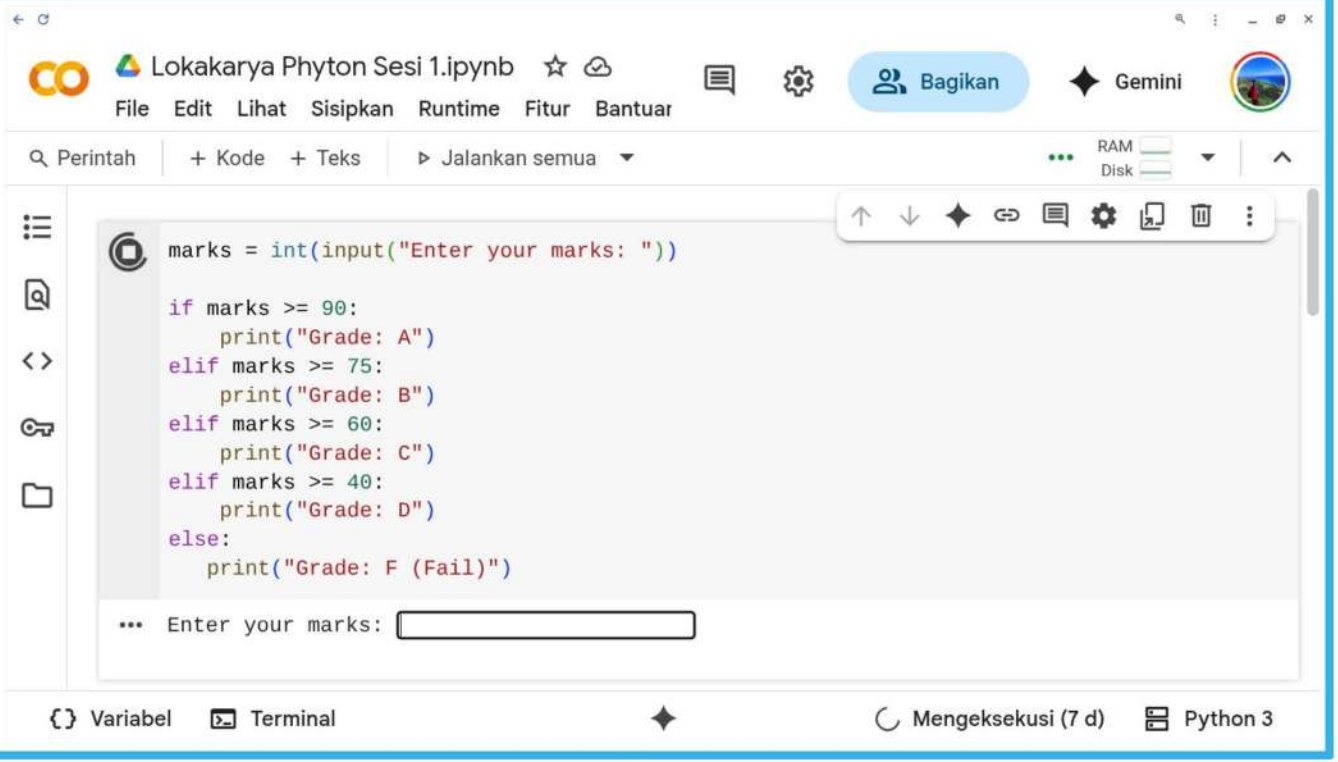
Sistem Penilaian:

```
marks = int(input("Enter your marks: "))

if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 60:
    print("Grade: C")
elif marks >= 40:
    print("Grade: D")
else:
   print("Grade: F (Fail)")
```

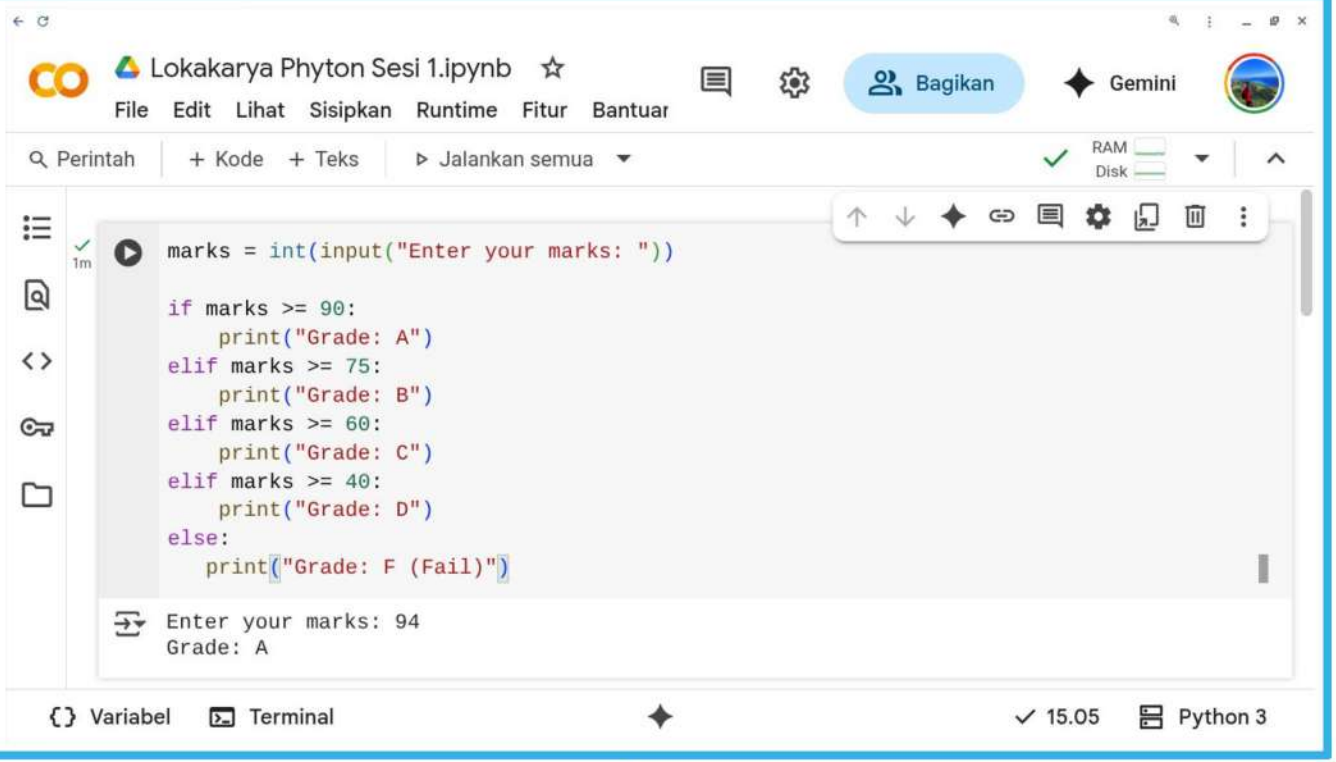
Contoh Penggunaan Pernyataan if-elif-else

Pengecekan Predikat Nilai



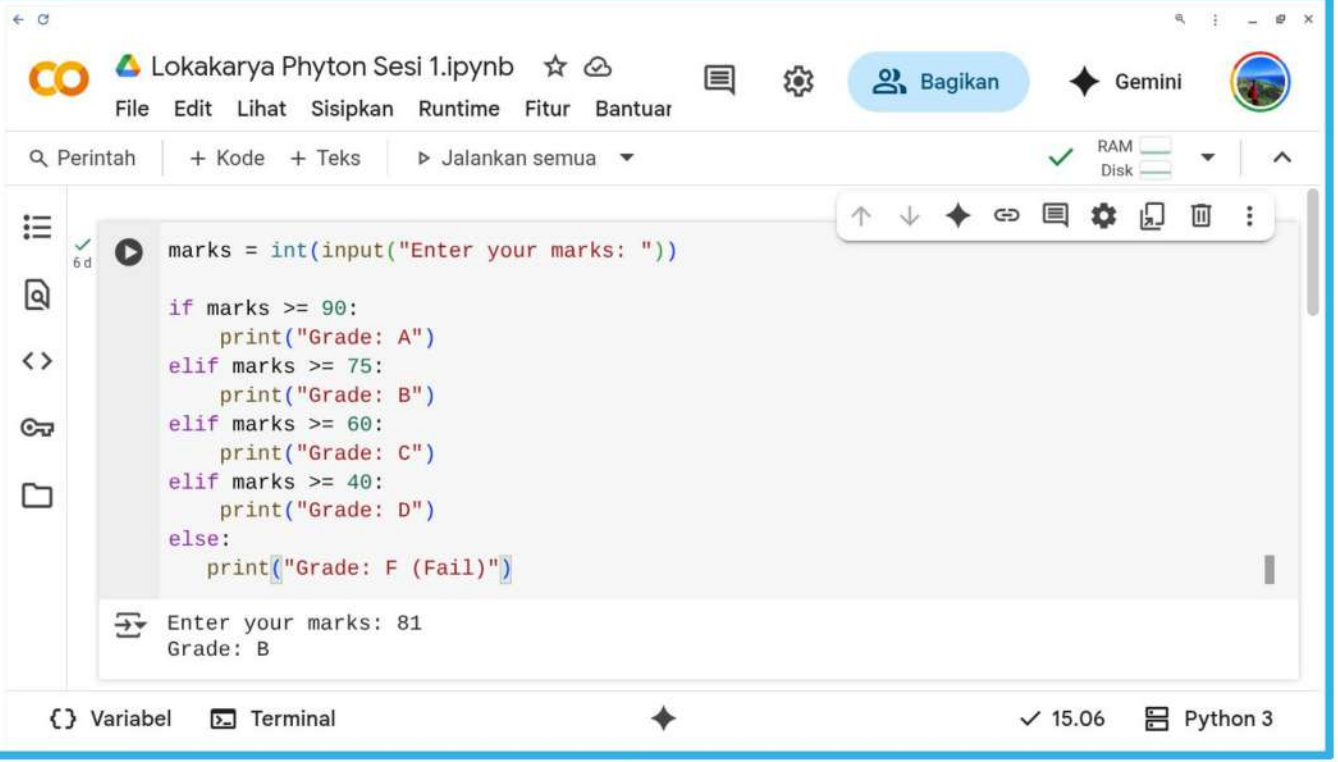
Contoh Penggunaan Pernyataan if-elif-else

Hasil Predikat
Nilai A



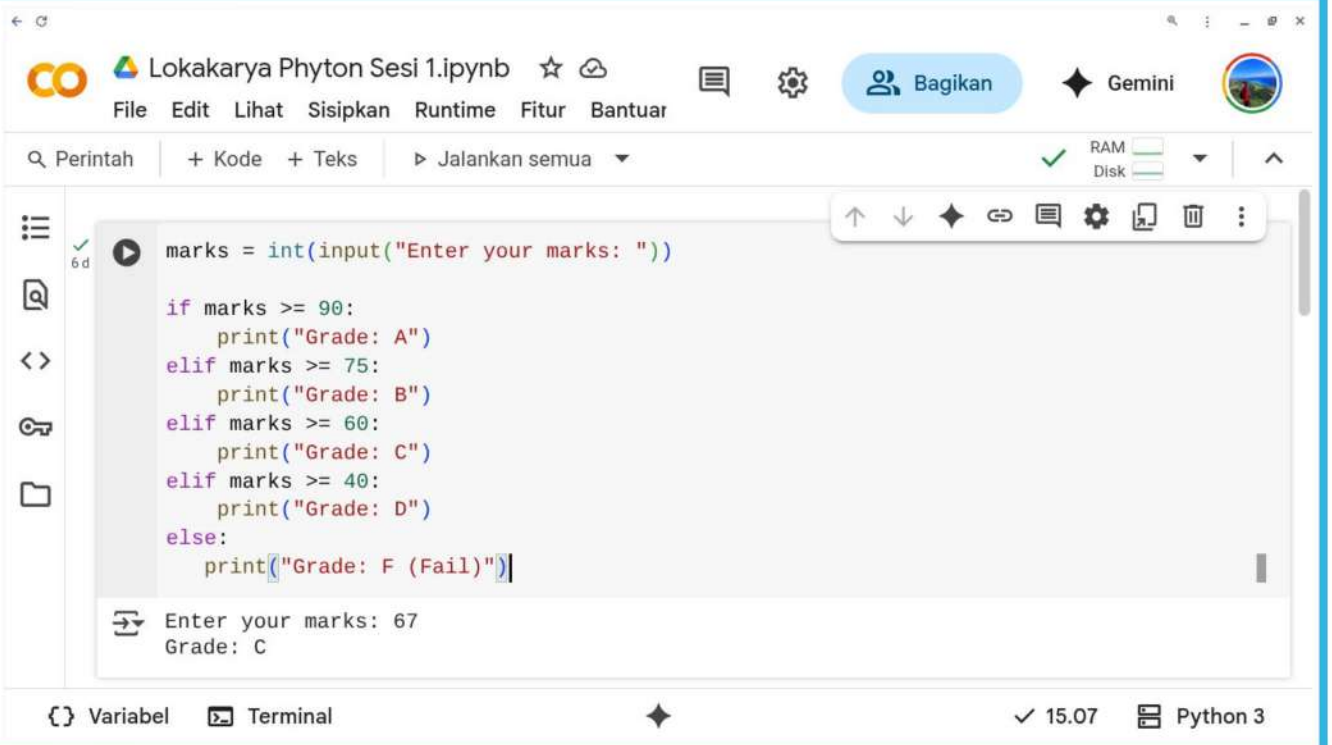
Contoh Penggunaan Pernyataan if-elif-else

Hasil Predikat
Nilai B



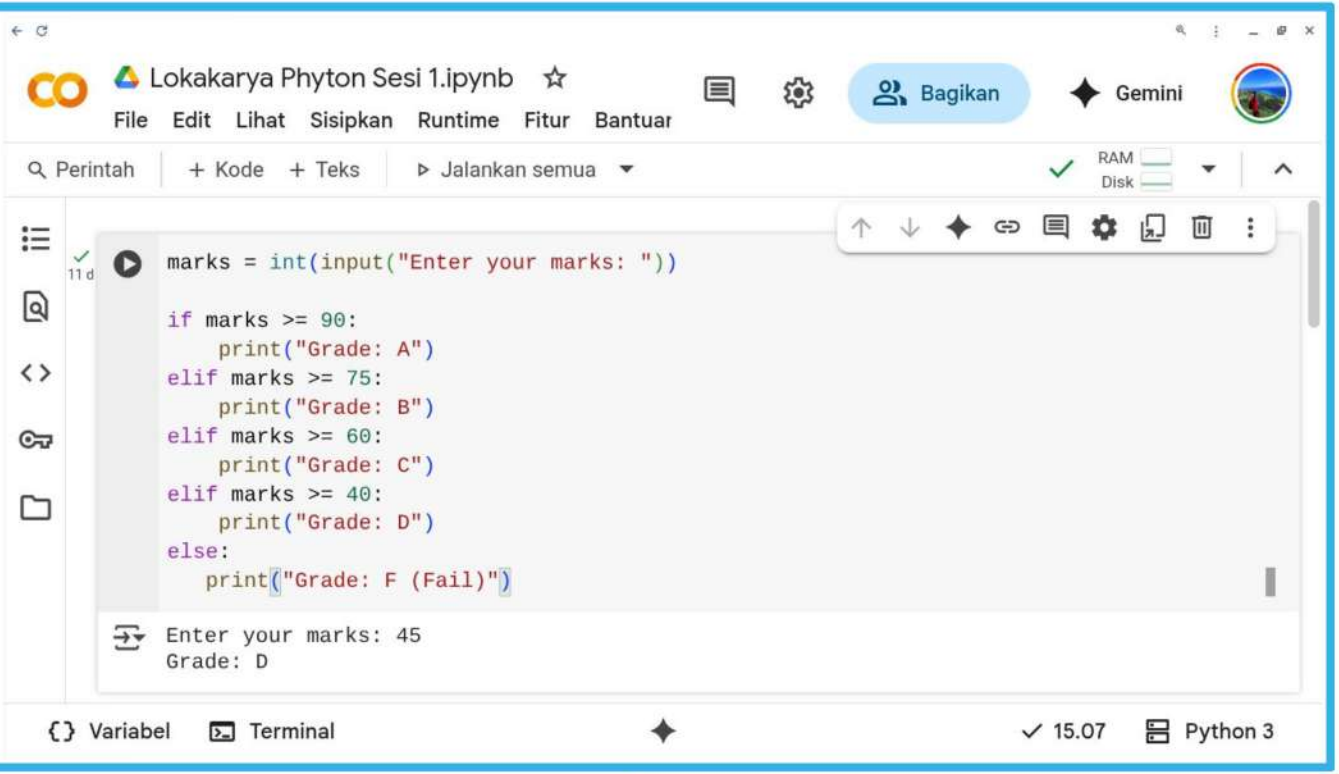
Contoh Penggunaan Pernyataan if-elif-else

Hasil Predikat
Nilai C



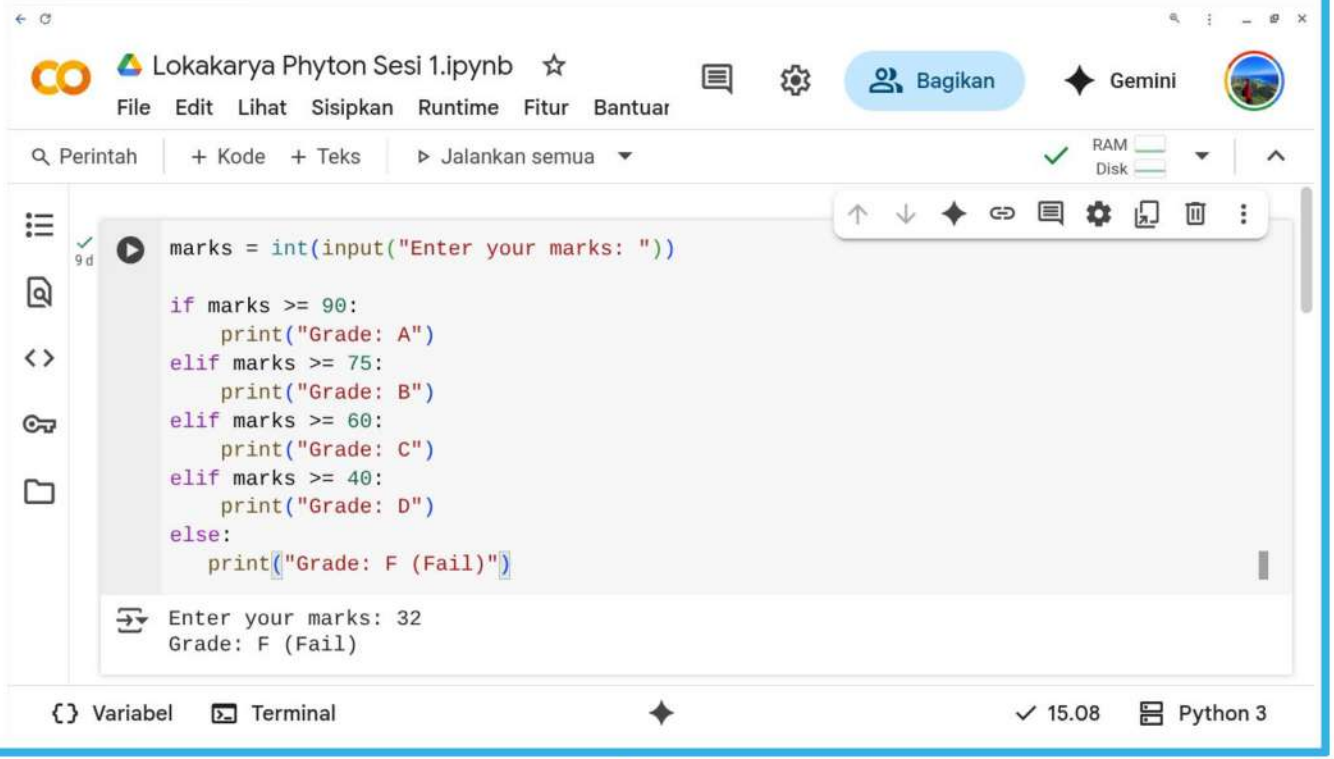
Contoh Penggunaan Pernyataan if-elif-else

Hasil Predikat
Nilai D



Contoh Penggunaan Pernyataan if-elif-else

Hasil Predikat
Nilai F



Contoh Penggunaan Pernyataan if-elif-else

Penjelasan:

- Jika nilainya , maka `90 or above` nilainya **A.**
- Jika nilainya , maka `75 or above but less than 90` nilainya **B.**
- Jika nilainya , maka `60 or above but less than 75` nilainya **C.**
- Jika nilainya , maka `40 or above but less than 60` nilainya **D.**
- Jika tidak, nilainya **F (Gagal)** .

Pernyataan Kondisional Bersarang

Pernyataan **kondisional bersarang** adalah pernyataan di dalam blok atau blok `if` lain . Ini berguna ketika keputusan bergantung pada beberapa kondisi. `if else`

Sintaksis:

```
if condition1:
    if condition2:
        # Code if both condition1 and condition2 are true
    else:
        # Code if condition1 is true but condition2 is false
else:
    # Code if condition1 is false
```




Contoh Penggunaan Pernyataan Kondisional Bersarang

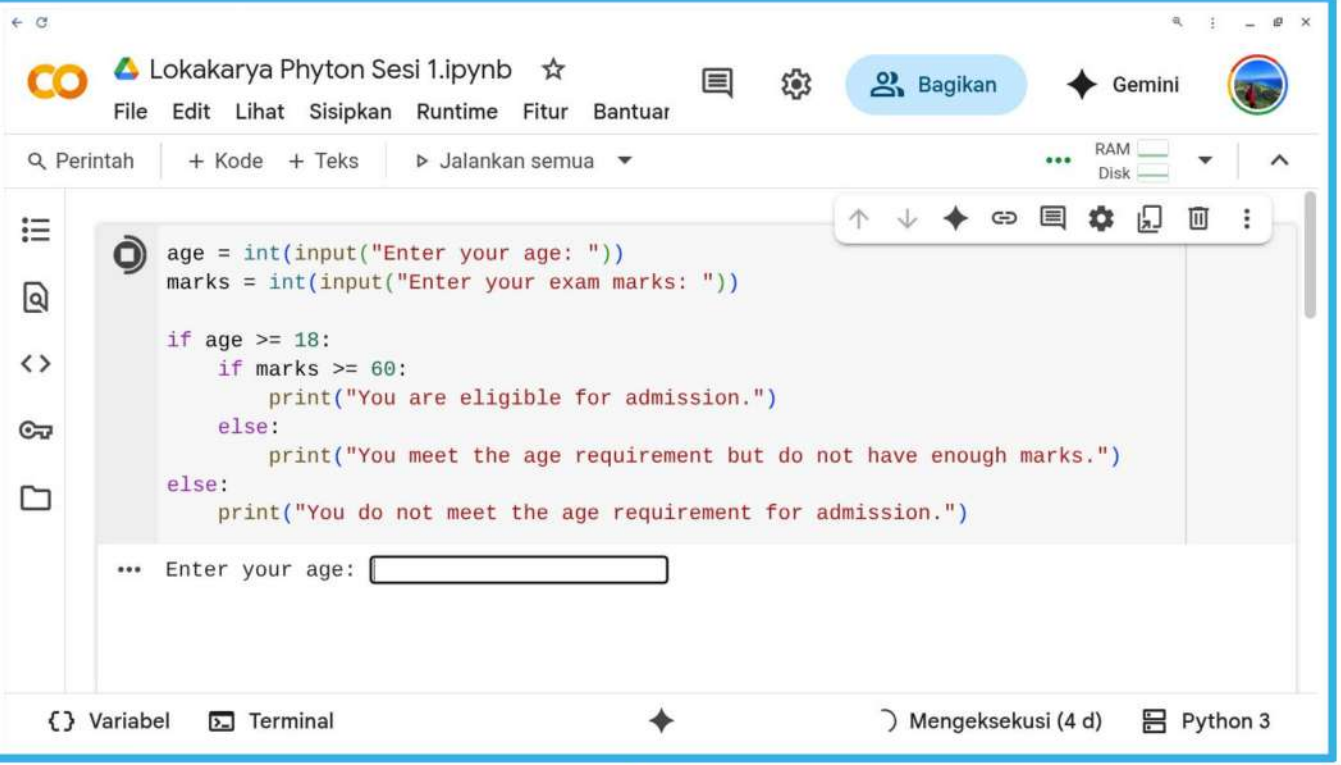
Memeriksa Kelayakan Masuk:

```
age = int(input("Enter your age: "))
marks = int(input("Enter your exam marks: "))

if age >= 18:
    if marks >= 60:
        print("You are eligible for admission.")
    else:
        print("You meet the age requirement but do not have enough
marks.")
else:
    print("You do not meet the age requirement for admission.")
```

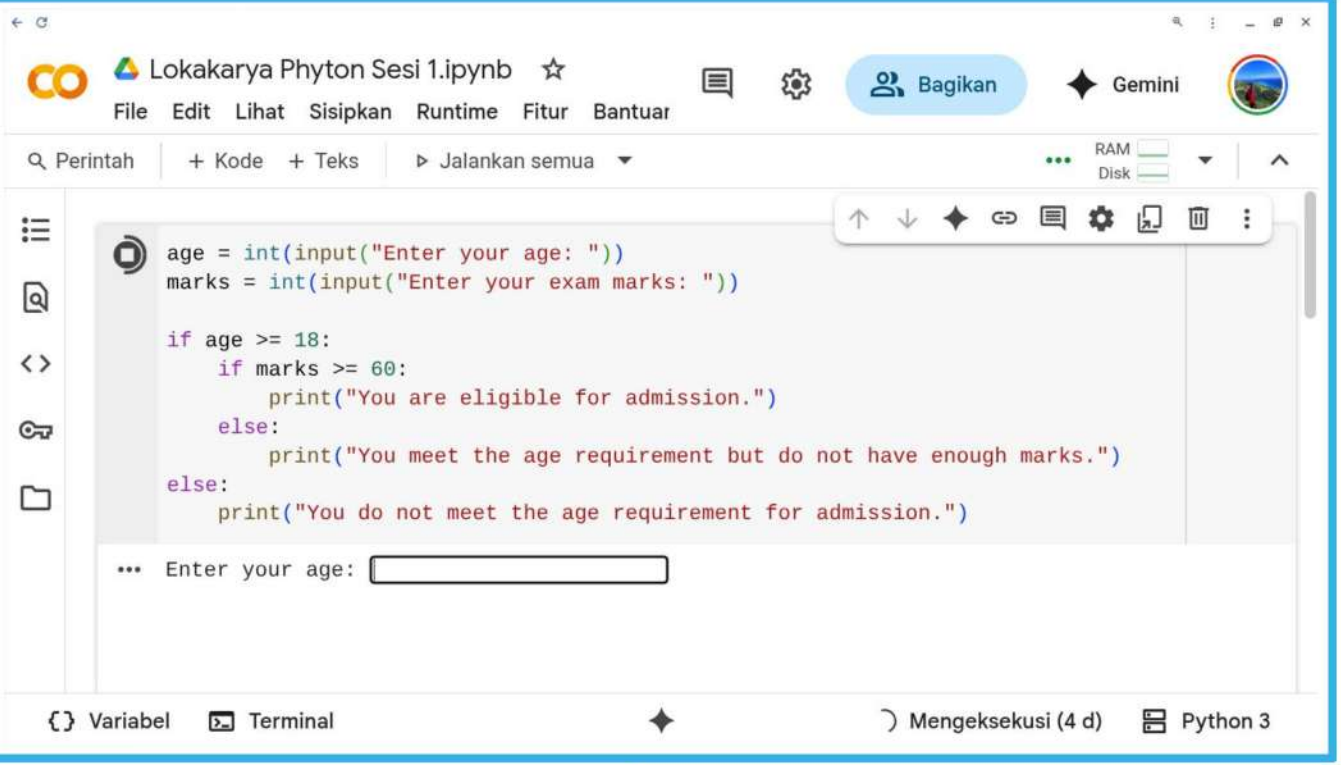
Contoh Penggunaan Pernyataan Kondisional Bersarang

Memeriksa kelayakan masuk kriteria



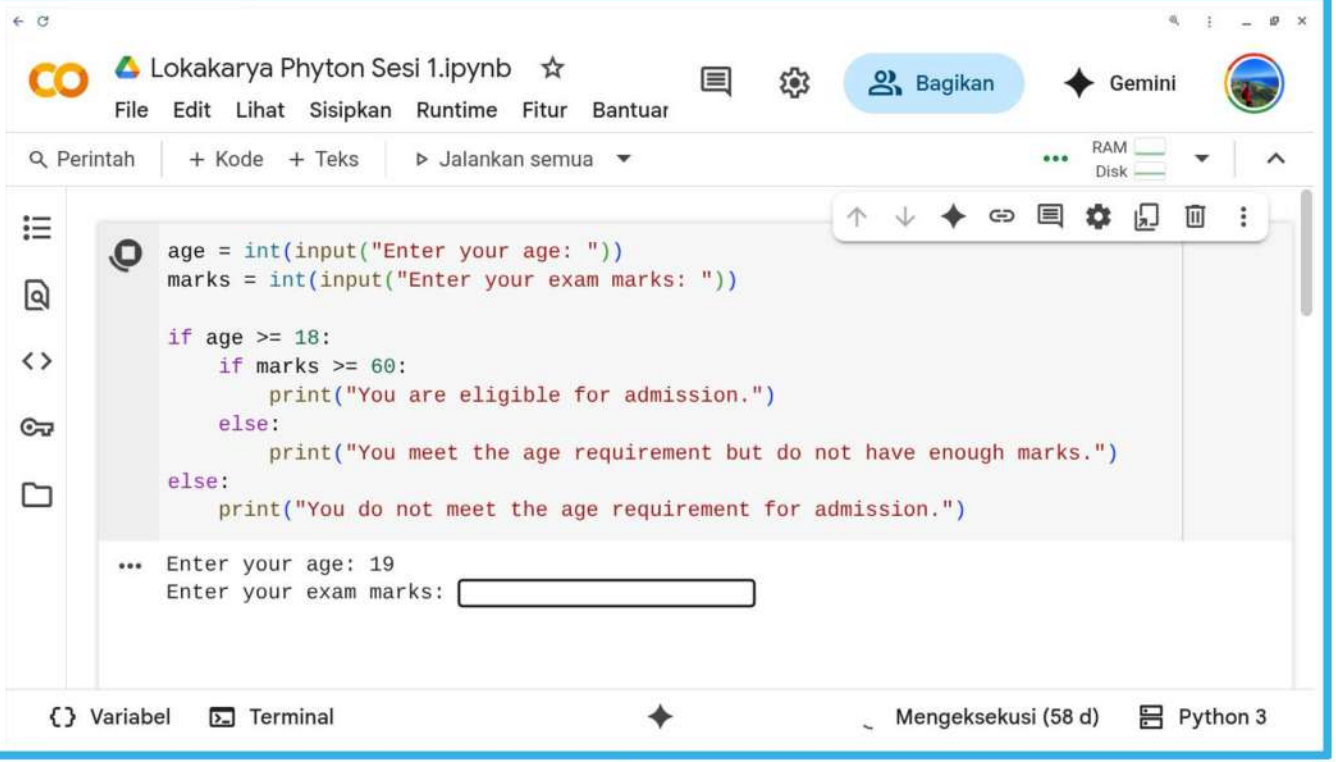
Contoh Penggunaan Pernyataan Kondisional Bersarang

Memeriksa kelayakan masuk kriteria



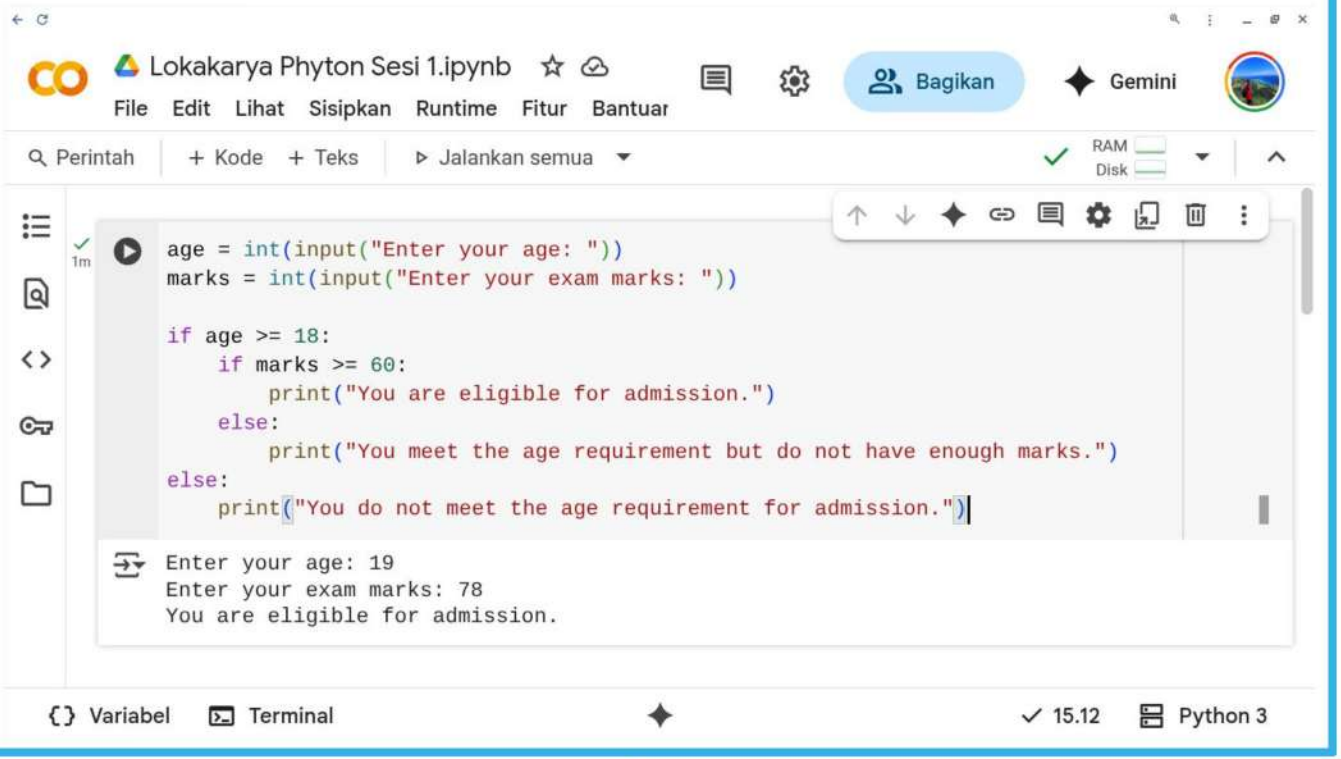
Contoh Penggunaan Pernyataan Kondisional Bersarang

Memasukkan Data Pertama



Contoh Penggunaan Pernyataan Kondisional Bersarang

Memasukkan Data Kedua



Contoh Penggunaan Pernyataan Kondisional Bersarang

Penjelasan

- Program ini pertama-tama memeriksa apakah pelamar berusia **18 tahun atau lebih** .
- Jika `True`, maka ia akan memeriksa apakah nilainya **60 atau lebih tinggi** .
- Jika kedua persyaratan terpenuhi, penerimaan diberikan.
- Jika usia sudah memenuhi syarat tetapi nilainya rendah, pengguna akan diberitahu tentang persyaratan nilai.
- Jika usianya di bawah 18 tahun, penerimaan akan langsung ditolak.

Operator Logika dalam Python

Python menyediakan operator logika untuk menggabungkan beberapa kondisi. Operator ini mengembalikan nilai `True` atau `False` berdasarkan kondisi yang diberikan.

Operator	Keterangan	Contoh	Hasil
`and`	Mengembalikan `True` jika **kedua** kondisinya `True`.	`(5 > 3) and (10 > 5)`	`True`
`or`	Mengembalikan `True Jika` **setidaknya satu** kondisi adalah `True.`	`(5 > 3) or (10 < 5)`	`True`
`not`	Membalikkan kondisi; mengembalikan `True Jika` kondisinya `False`	`not(5 > 3)`	`False`

Contoh Operator Logika dalam Python

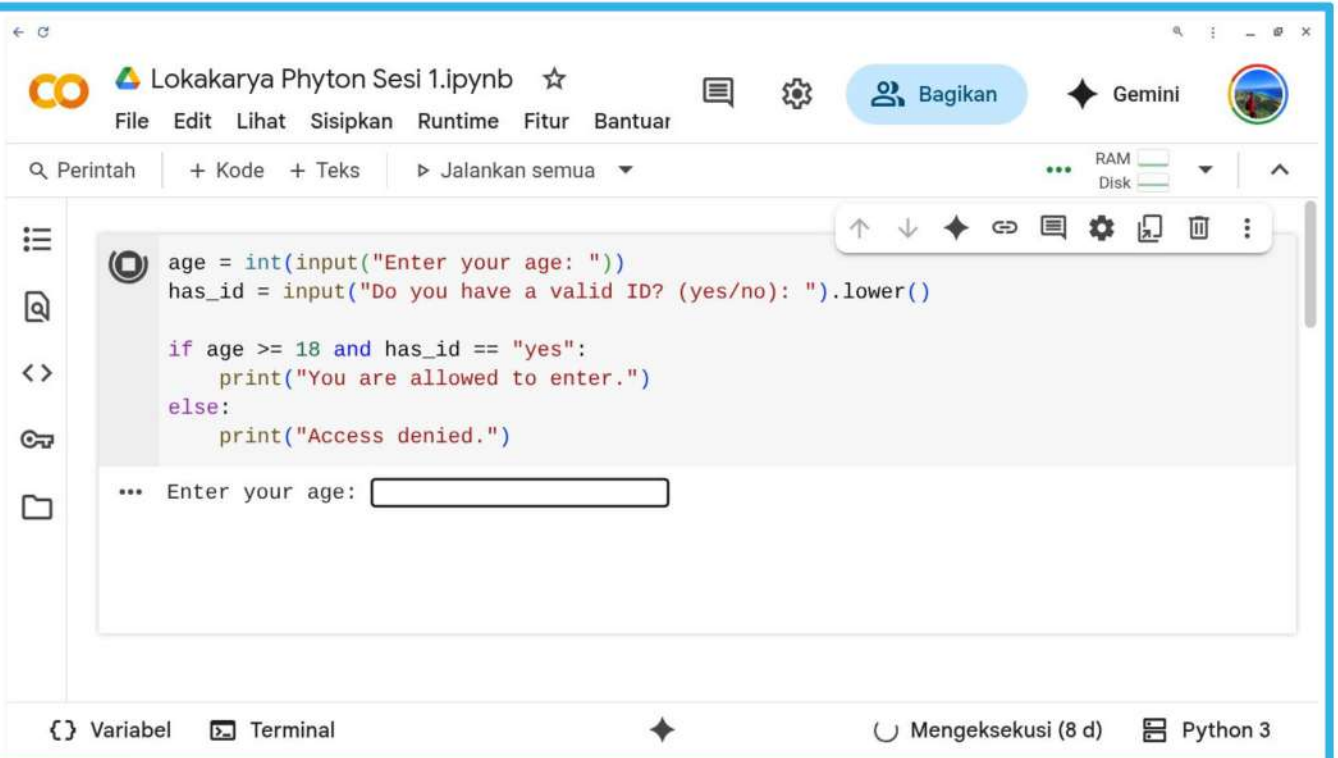
Operator Logika dalam Tindakan :

```
age = int(input("Enter your age: "))
has_id = input("Do you have a valid ID? (yes/no):
").lower()

if age >= 18 and has_id == "yes":
    print("You are allowed to enter.")
else:
    print("Access denied.")
```

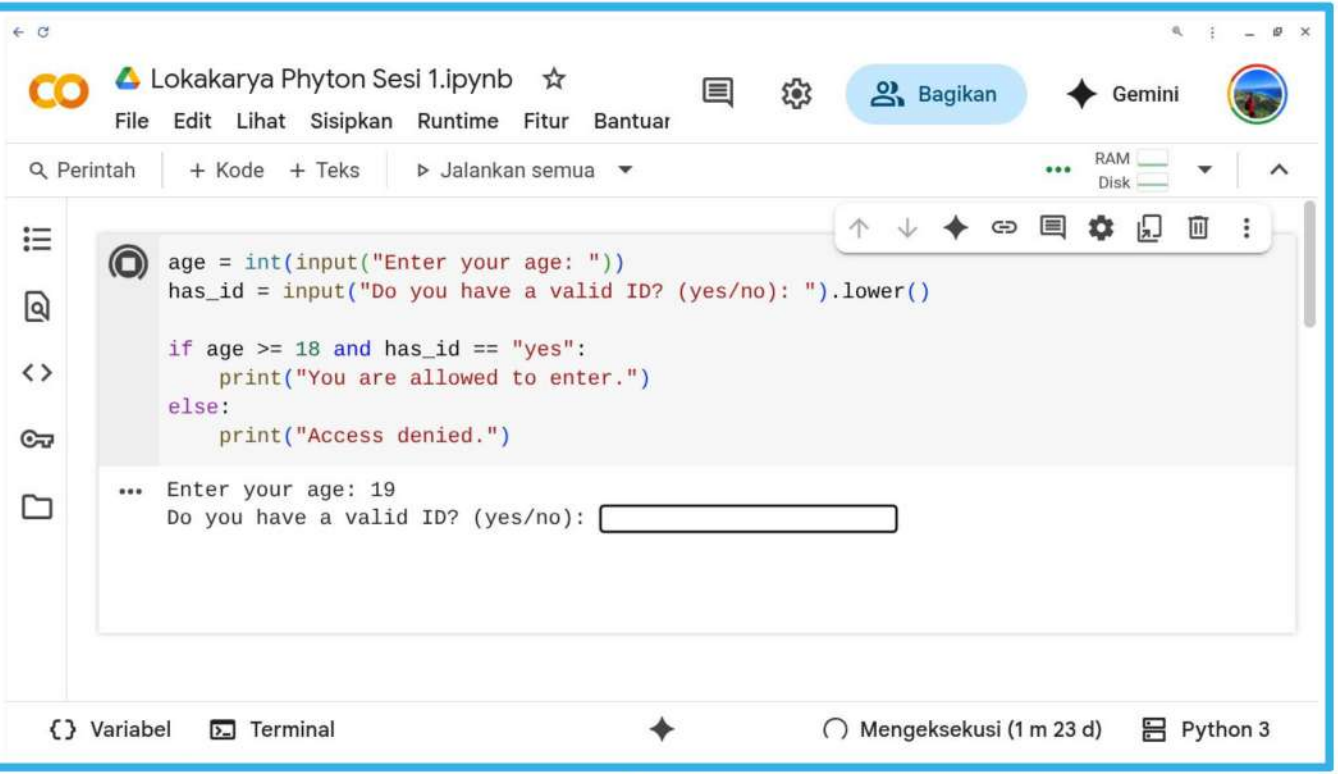
Contoh Operator Logika dalam Python

Memasukkan Data Usia



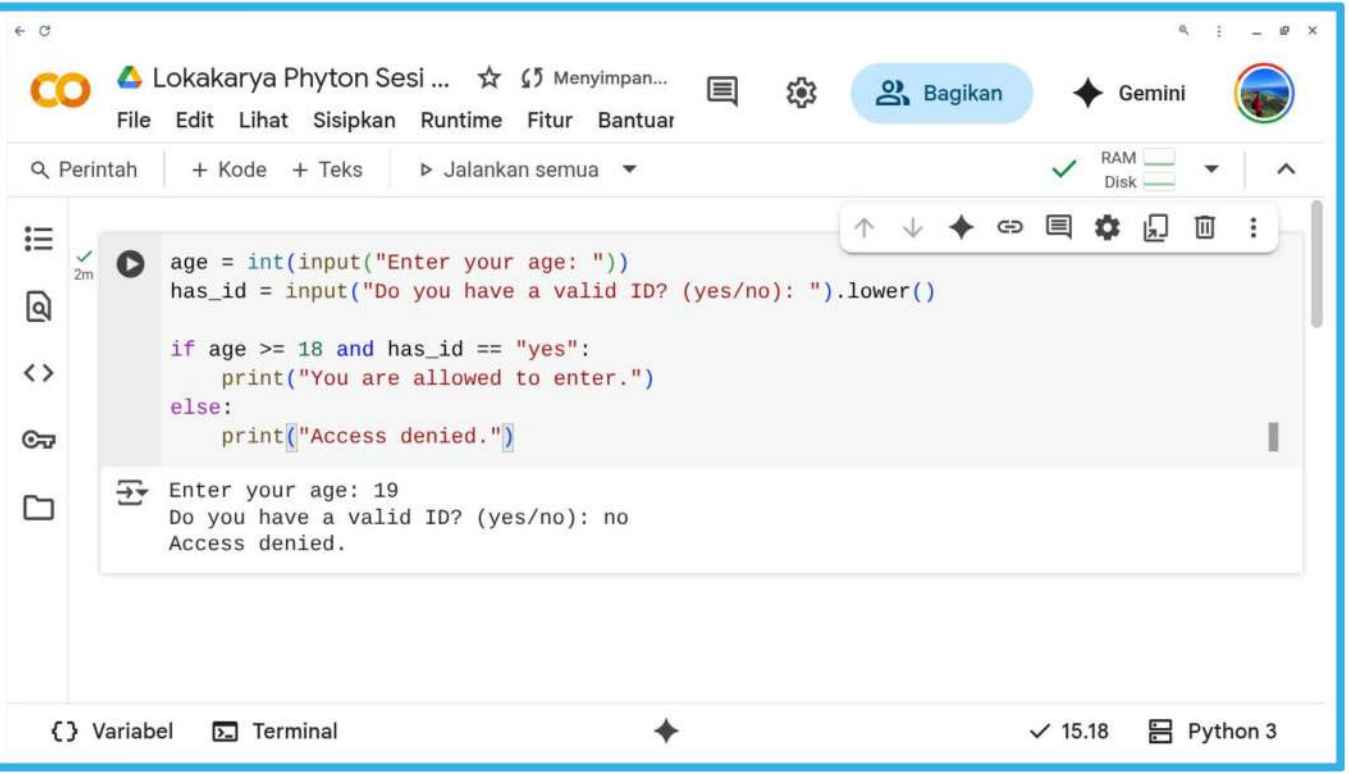
Contoh Operator Logika dalam Python

Memasukkan Data Kepemilikan Identitas



Contoh Operator Logika dalam Python

Hasil Pengecekan



Contoh Operator Logika dalam Python

Penjelasan:

- Pengguna harus berusia **minimal 18 tahun dan** harus memiliki **ID yang valid** (`"yes"`).
- Jika kedua kondisi terpenuhi `True`, akses diberikan.
- Jika salah satu kondisinya `False`, akses ditolak.

Aktivitas: Kalkulator BMI

Kita akan membuat Kalkulator BMI (Indeks Massa Tubuh) menggunakan Python. Program ini akan:

- Ambil berat (dalam kilogram) dan tinggi (dalam meter) pengguna sebagai masukan.
- Hitung BMI menggunakan rumus standar.
- Kategorikan hasil BMI berdasarkan rentang yang telah ditentukan sebelumnya.
- Berikan umpan balik kepada pengguna tentang kategori BMI mereka.

Memahami BMI

BMI (Indeks Massa Tubuh) adalah perhitungan sederhana yang digunakan untuk menentukan apakah seseorang memiliki berat badan ideal sesuai tinggi badannya. Rumusnya adalah:

BMI = berat badan (kg) / tinggi badan (m)^2

Kisaran BMI	Kategori
Kurang dari 18,5	Berat badan kurang
18,5 – 24,9	Berat Badan Normal
25 – 29,9	Kegemukan
30 atau lebih	Kegemukan

Menulis Program Kalkulator BMI

Mengambil Masukan Pengguna

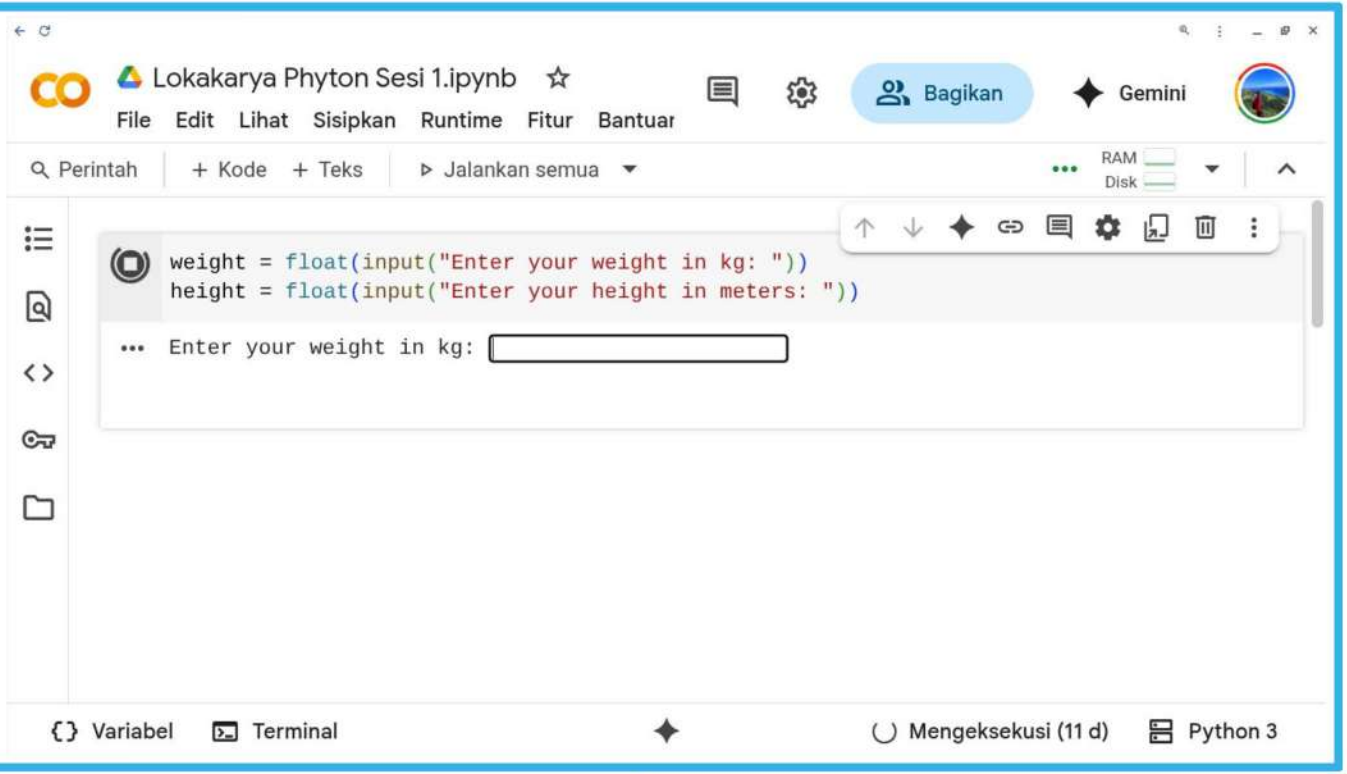
- Program ini pertama-tama meminta pengguna untuk memasukkan **berat badan dalam kilogram** dan **tinggi badan dalam meter** .
- Fungsi ini `input()` digunakan untuk menerima input berupa string, dan `float()` digunakan untuk mengubahnya menjadi angka desimal untuk perhitungan.

```
weight = float(input("Enter your weight in kg: "))
```

```
height = float(input("Enter your height in meters: "))
```

Menulis Program Kalkulator BMI

Inputan Data Secara Bertahap



Menulis Program Kalkulator BMI

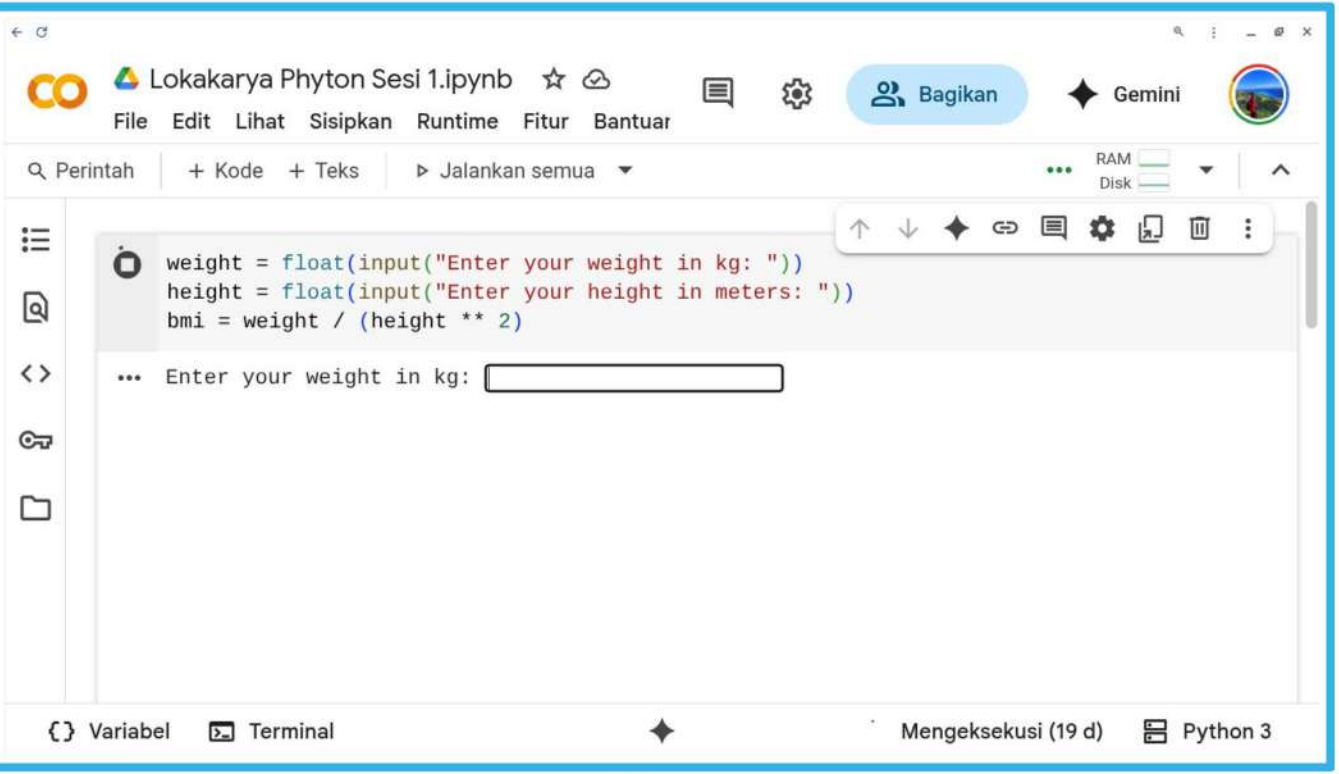
Menghitung BMI

- Rumus BMI diterapkan menggunakan operator matematika Python:
- `height ** 2` menghitung kuadrat tinggi.
- Divisi ini `/` menghitung BMI.

```
bmi = weight / (height ** 2)
```

Menulis Program Kalkulator BMI

Persamaan BMI



Menulis Program Kalkulator BMI

Menggunakan Pernyataan Kondisional untuk Mengklasifikasikan BMI

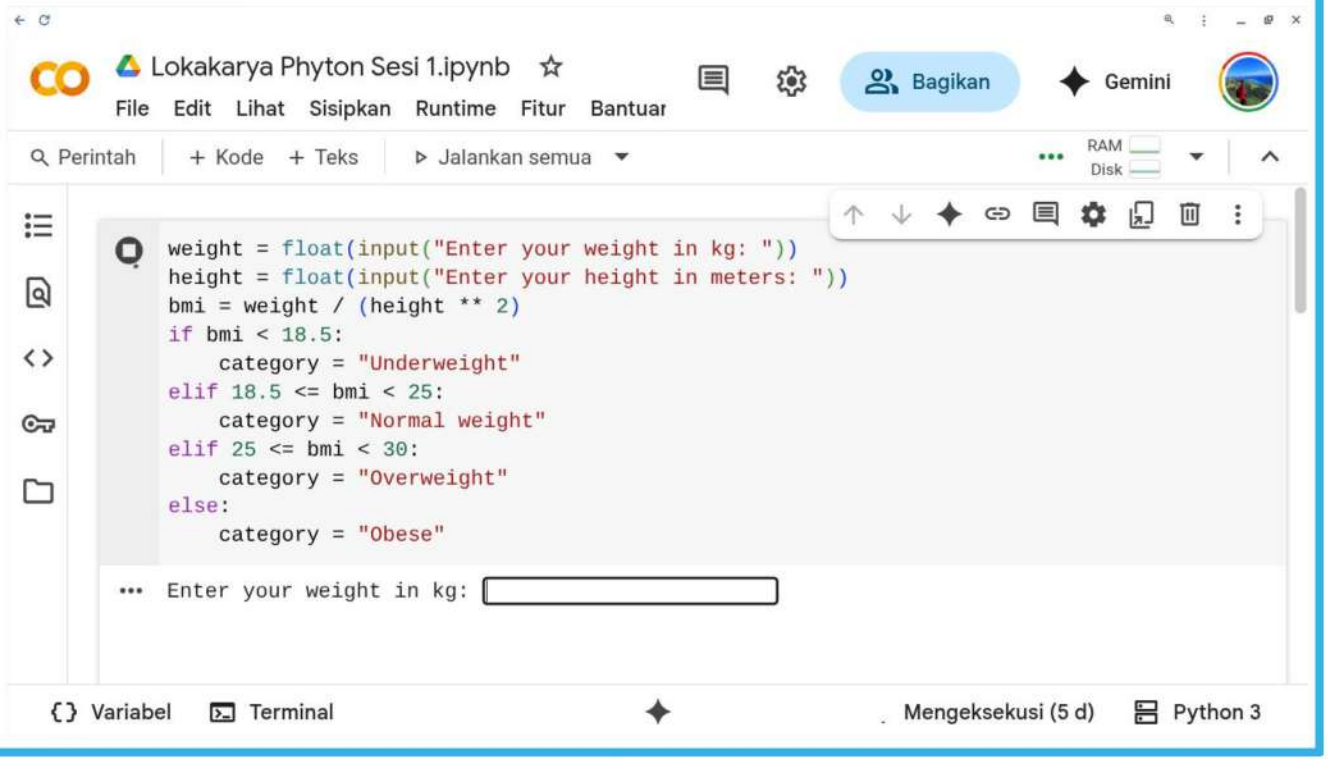
- Program ini memeriksa nilai BMI terhadap berbagai kategori menggunakan pernyataan `if`, `elif`, dan `else`
- Jika BMI **kurang dari 18,5** , pengguna diklasifikasikan sebagai **kekurangan berat badan** .
- Jika BMI **antara 18,5 dan 24,9** , pengguna diklasifikasikan sebagai **berat badan normal** .
- Jika BMI **antara 25 dan 29,9** , pengguna diklasifikasikan sebagai **Kegemukan** .
- Jika BMI **30 atau lebih** , pengguna diklasifikasikan sebagai **Obesitas** .

Menulis Program Kalkulator BMI

```
if bmi < 18.5:
    category = "Underweight"
elif 18.5 <= bmi < 25:
    category = "Normal weight"
elif 25 <= bmi < 30:
    category = "Overweight"
else:
    category = "Obese"
```

Menulis Program Kalkulator BMI

Memasukan Kondisi Kriteria BMI



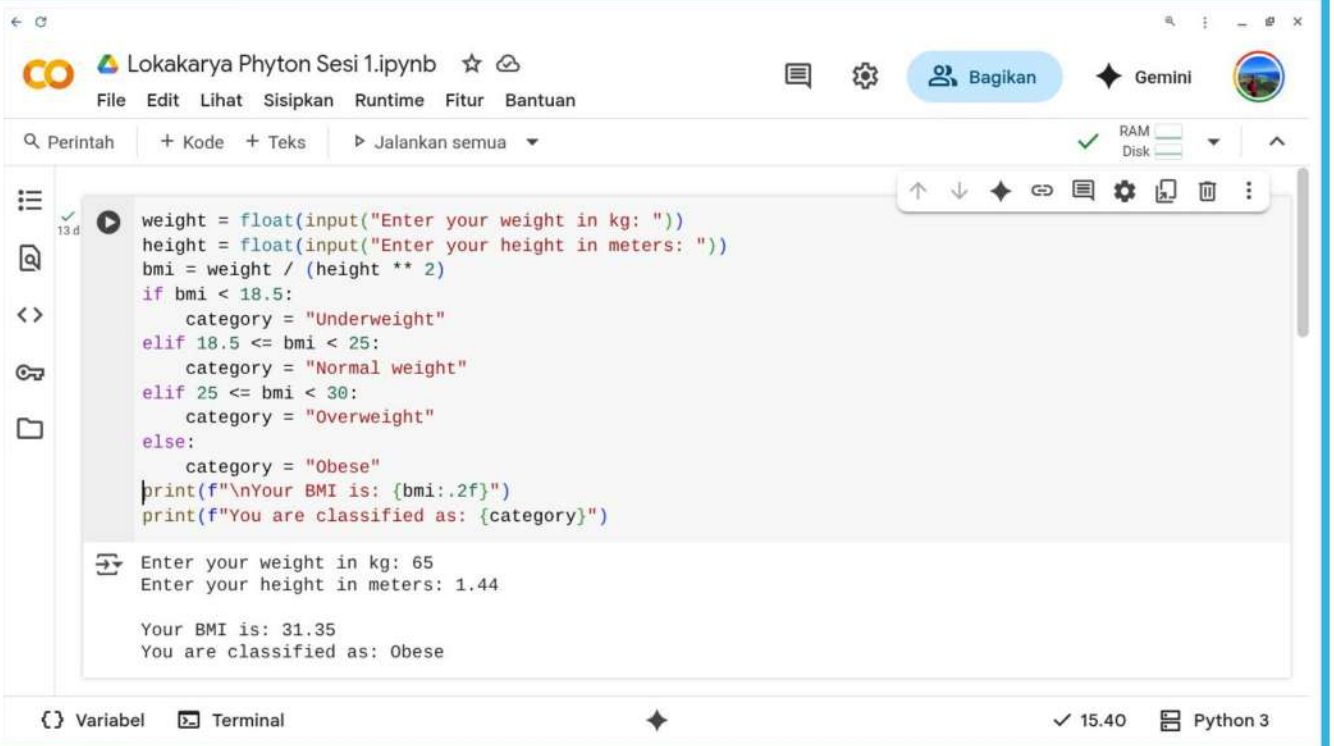
Menampilkan Hasil

- Program ini mencetak nilai BMI hingga **dua tempat desimal** menggunakan `:.2f` penentu format.
- Kemudian mencetak kategori yang sesuai.

```
print(f"\nYour BMI is: {bmi:.2f}")
print(f"You are classified as: {category}")
```

Menulis Program Kalkulator BMI

Hasil setelah data usia dan tinggi dimasukkan



Aktivitas: Permainan Tebak Angka

Dalam aktivitas ini, kita akan membuat **Game Tebak Angka** menggunakan Python. Game ini akan:

- Hasilkan angka acak antara **1 dan 20** .
- Berikan pemain **lima kesempatan** untuk menebak angka yang benar.
- Berikan petunjuk jika tebakannya terlalu tinggi atau terlalu rendah.
- Menampilkan pesan yang menunjukkan apakah pemain **menang atau kalah** .

Memahami Logika Permainan

1. Program ini **secara acak memilih angka** antara **1 dan 20** .
2. Pemain memasukkan tebakan.
3. Program memeriksa tebakan:
   a. Jika tebakannya **benar** , pemain menang.
   b. Jika tebakannya **terlalu rendah** , program akan memberi tahu pemain.
   c. Jika tebakannya **terlalu tinggi** , program akan memberi tahu pemain.
4. Pemain mendapat **lima kesempatan** untuk menebak angkanya.
5. Jika pemain **gagal menebak** dalam lima kali percobaan, program **akan menampilkan nomornya** .

Menulis Program Permainan Tebak Angka

1. Mengimpor `random` Modul

- Kita perlu `random` modul untuk menghasilkan angka acak.

```
import random
```

2. Menghasilkan Angka Acak

- `random.randint(1, 20)` menghasilkan bilangan bulat acak antara **1 dan 20** .

```
number_to_guess = random.randint(1, 20)
```

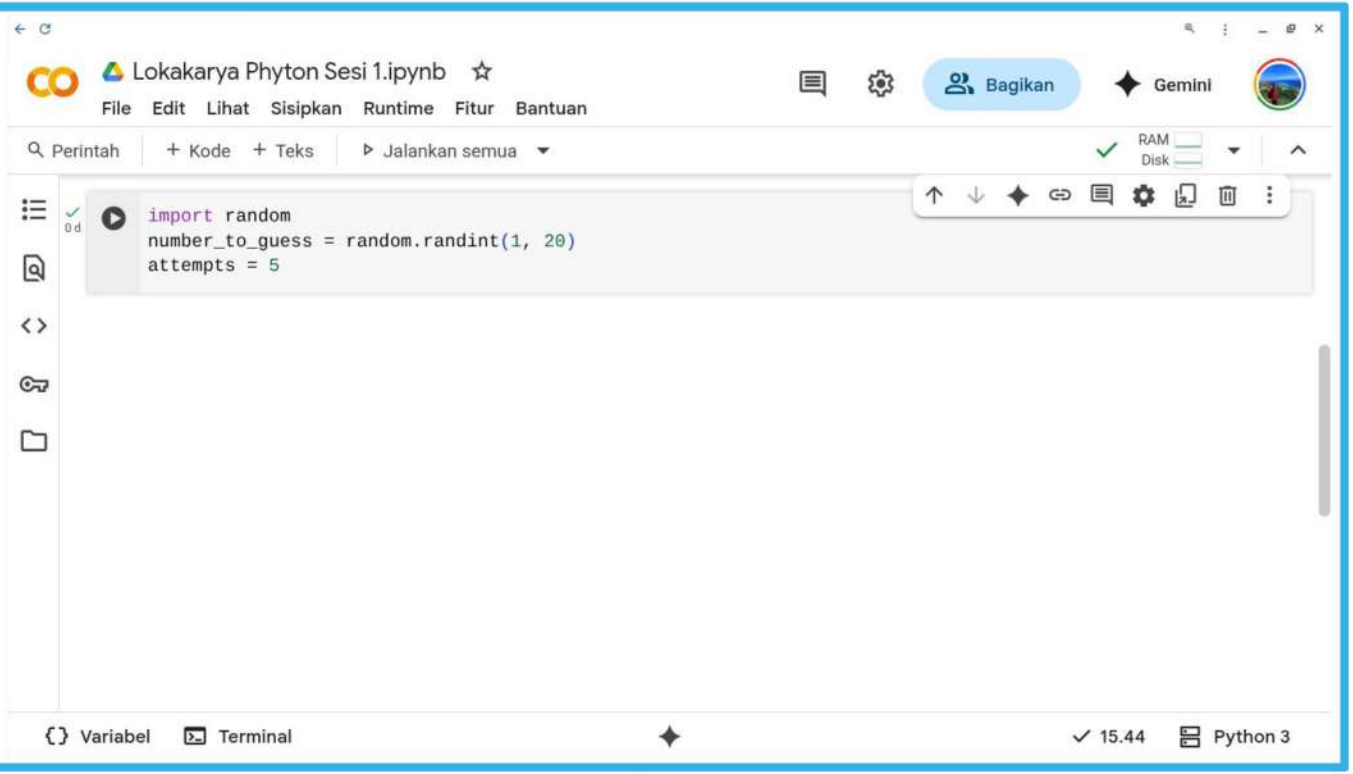
3. Mengatur Jumlah Percobaan

- Pemain diberi **lima kesempatan** untuk menebak angkanya.

```
attempts = 5
```

Menulis Program Permainan Tebak Angka

Impor Library dan menetapkan rentang angka serta kesempatan



Menulis Program Permainan Tebak Angka

4. Memulai Permainan

- Program ini mencetak pesan untuk memberi tahu pengguna tentang permainan.

```
import random
number_to_guess = random.randint(1, 20)
attempts = 5
print("Guess the number (between 1 and 20)!")
```

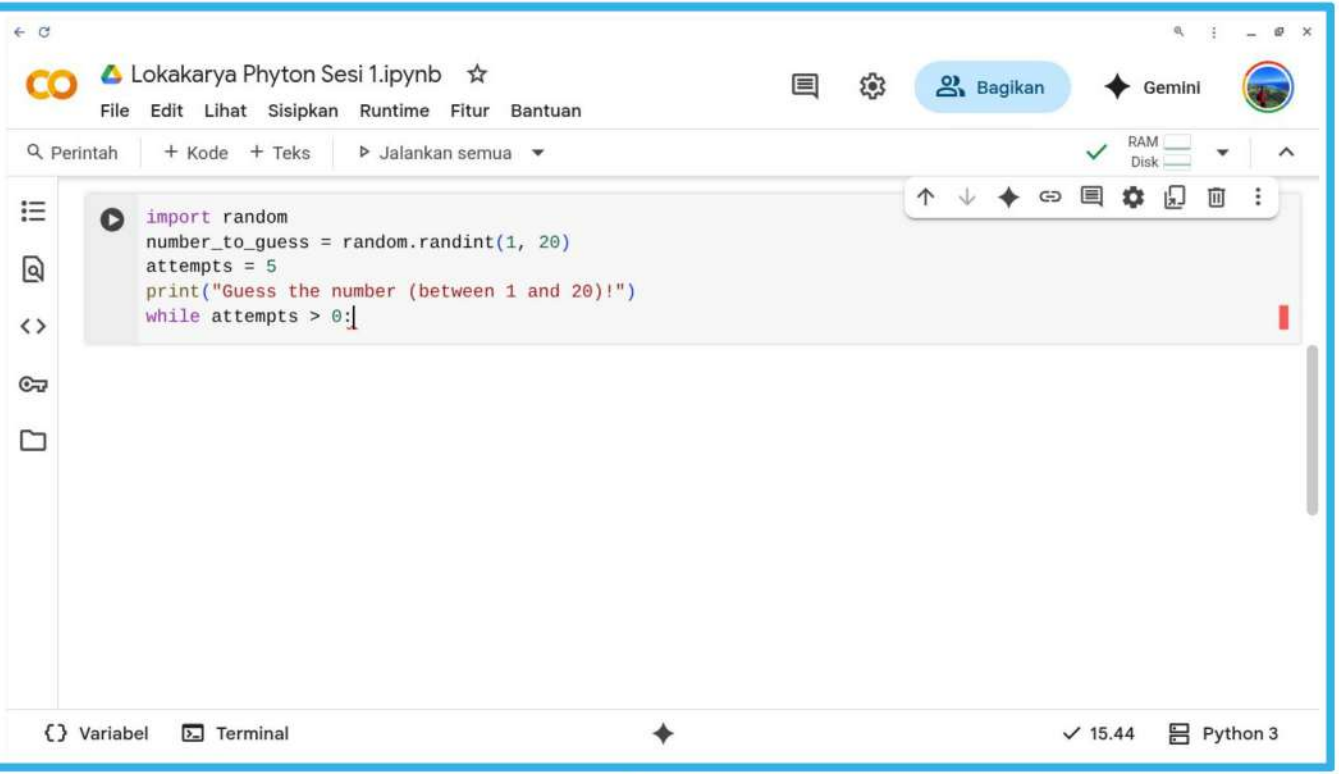
5. Menjalankan Loop untuk Tebakan Pengguna

- Perulangan ini terus berlanjut **hingga pemain kehabisan percobaan** .

```
while attempts > 0:
```

Menulis Program Permainan Tebak Angka

Cetak petunjuk dan gunakan perulangan



Menulis Program Permainan Tebak Angka

6. Mengambil Masukan dari Pemain

- Pengguna memasukkan tebakannya, dan `int(input())` mengubahnya menjadi bilangan bulat.

```
guess = int(input("Enter your guess: "))
```

7. Memeriksa Tebakan Menggunakan Pernyataan Kondisional

- Program ini **membandingkan angka tebakan** dengan angka sebenarnya menggunakan kondisi `if`, `elif`, dan `else`
- Jika tebakannya **benar** , pemain menang, dan program keluar menggunakan `break`.

Menulis Program Permainan Tebak Angka

Ambil inputan angka



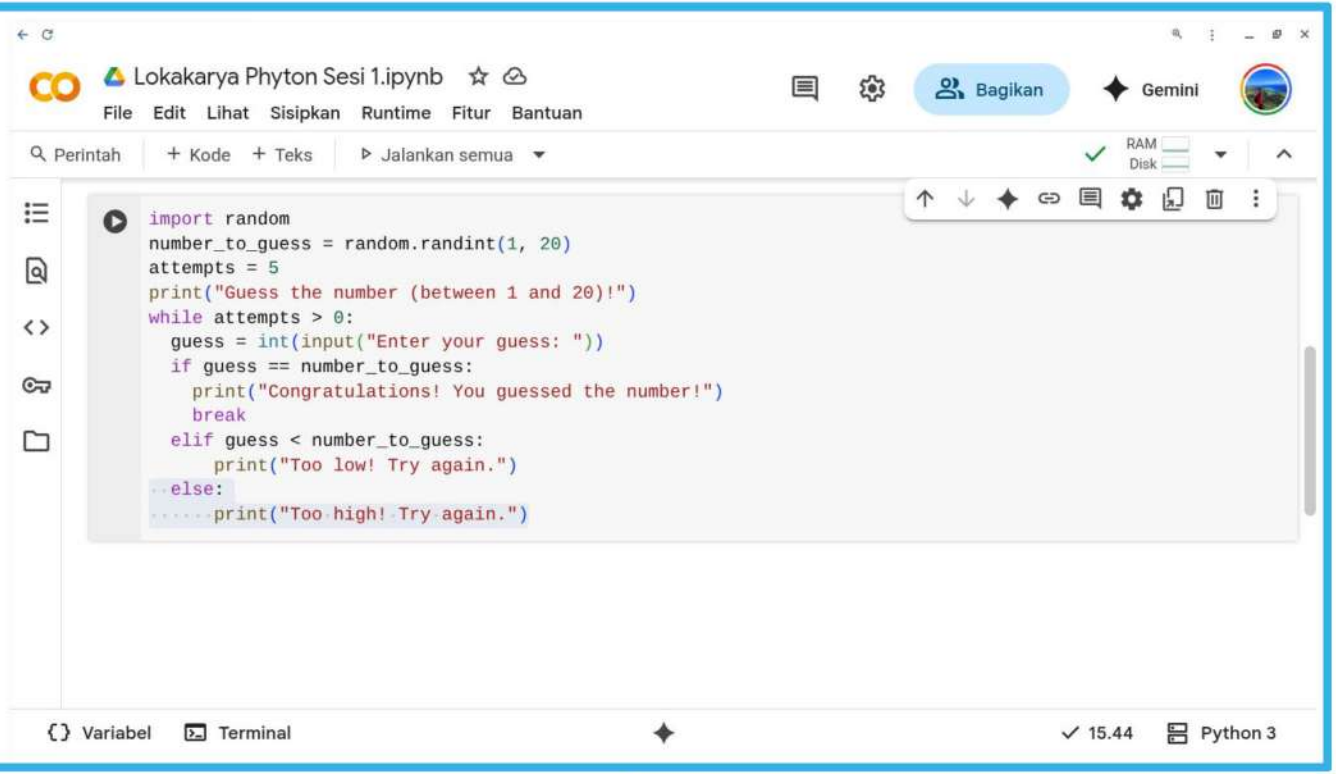
Menulis Program Permainan Tebak Angka

- Jika tebakannya **terlalu rendah** , program akan meminta pemain untuk menebak **angka yang lebih tinggi** .
- Jika tebakannya **terlalu tinggi** , program akan memberitahu pemain untuk menebak **angka yang lebih rendah** .

```
if guess == number_to_guess:
    print("Congratulations! You guessed the number!")
    break
elif guess < number_to_guess:
    print("Too low! Try again.")
else:
    print("Too high! Try again.")
```

Menulis Program Permainan Tebak Angka

Masukkan Kondisi Pengecekan



Menulis Program Permainan Tebak Angka

8. Mengurangi Jumlah Percobaan

- Setelah setiap tebakan salah, jumlah percobaan **berkurang sebanyak 1** .

```
attempts -= 1
print(f"Attempts left: {attempts}")
```

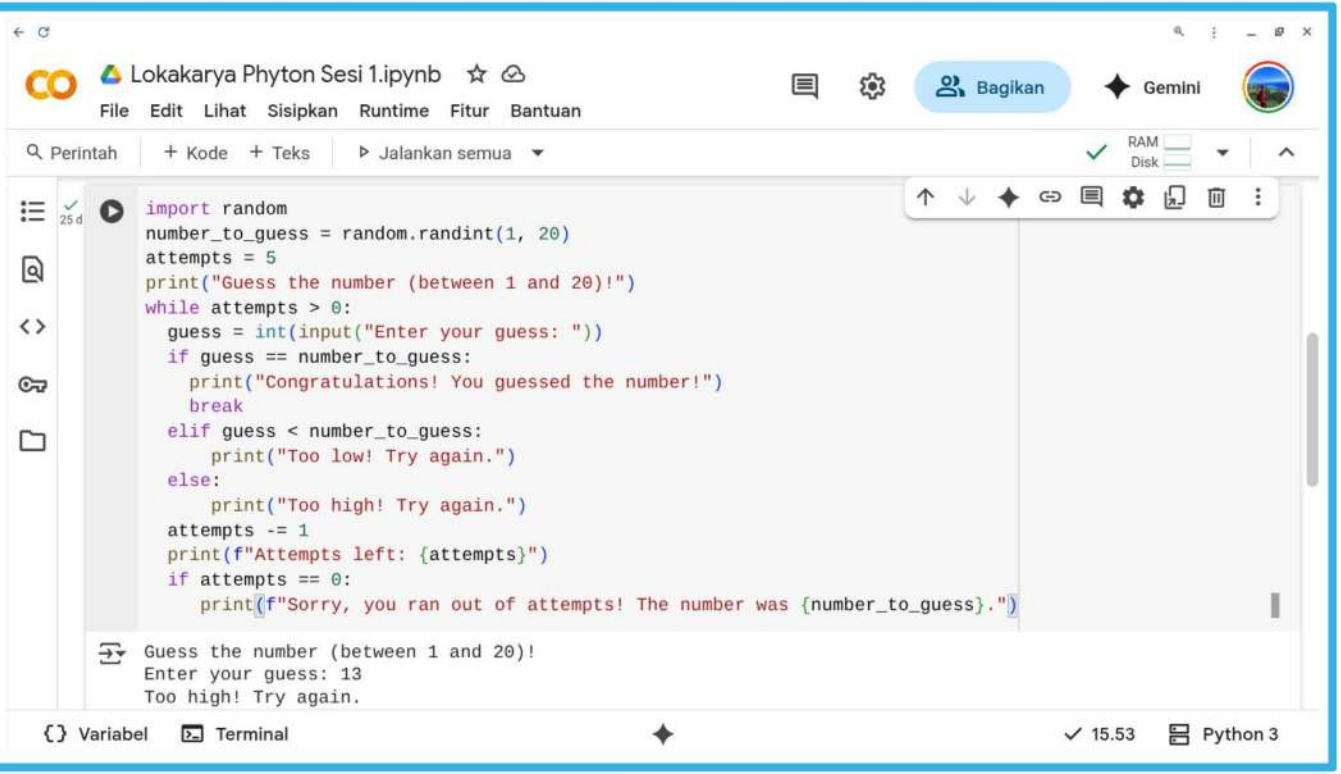
9. Mengakhiri Permainan

- Jika pemain **kehabisan percobaan** , permainan akan menampilkan angka yang benar.

```
if attempts == 0:
    print(f"Sorry, you ran out of attempts! The number was {number_to_guess}.")
```

Menulis Program Permainan Tebak Angka

Hasil Permainan



Tantangan Python

1. **Buat kalkulator sederhana**
   Buat kalkulator yang dapat melakukan operasi matematika dasar seperti penjumlahan, pengurangan, perkalian, dan pembagian berdasarkan masukan pengguna.
2. **Konverter Suhu**
   Buatlah program yang mengubah suhu antara Celsius dan Fahrenheit.
3. **Permainan Tebak Angka:**
   Buat permainan di mana komputer memilih angka antara 1 dan 100, dan pemain harus menebaknya. Berikan umpan balik jika tebakannya terlalu tinggi atau terlalu rendah.
4. **Kalkulator Usia**
   Tulis program di mana pengguna memasukkan tahun kelahiran mereka dan program menghitung usia mereka.
5. **Pemeriksa Genap atau Ganjil**
   Buatlah program yang meminta angka kepada pengguna dan mencetak apakah angka tersebut genap atau ganjil.